



SDT621

FORMATIVE ASSESSMENT 2



Josh Jansen Van Rensburg

Contents

Declaration Of Authenticity.....	3
Task 1.....	4
Question 1.1	4
Question 1.2	4
Question 1.3	5
Task 2.....	6
Task 3.....	8
Test Case 1:	8
Test Case 2:	9
Test Case 3:	10
Test Case 4:	12
Test Case 5:	13
Test Case 6:	14
Test Case 7:	15
Test Case 8:	16
Test Case 9:	17
Test Case 10:	18
Test Case 11:	19
Test Case 12:	20
Task 4.....	21
Task 5.....	24
Defect 1	24
Defect 2.....	25
Defect 3.....	26
Defect 4.....	28
Defect 5.....	29
Task 6.....	31
Task 7.....	32
SOFTWARE TESTING REPORT – ULMS	32
Executive Summary	32
Testing Scope	32

Tools Used	32
Defects Identified	32
Corrections Implemented	33
Recommendations	33
Deployment Readiness Decision	33

Declaration Of Authenticity

Please complete the declaration of authenticity below for all assignments:

DECLARATION OF AUTHENTICITY

I Josh Jansen Van Rensburg hereby

(FULL NAME)

declare that the contents of this assignment are entirely my own work with the exception of the following elements: (List the elements of work in this project that were not self-generated as well as who the originator of the element is)

Element	Originator

Signature:  Date: 05/03/2026

Task 1

Question 1.1

System Purpose: ULMS is a cloud-enabled Learning Management System supporting student registrations, login authentication, course enrolment, marks capture, and report generation for Umoja Group.

Testing Goals:

- Validate all functional requirements (login, enrolment, marks, reports)
- Identify and document all defects
- Verify non-functional requirements (performance, usability, security)
- Ensure error handling and validation rules work correctly
- Confirm average calculation accuracy
- Validate duplicate prevention in course enrolment

Scope Of Testing: Login validation, student registration form, course enrolment rules, marks capture, report generation, navigation, error messages, load testing beyond 100 concurrent users, database stress testing, third-party integrations, mobile responsiveness

Question 1.2

Testing Type	Approach	Example Test Scenario
Functional Testing	Black-box testing using equivalence partitioning & boundary value analysis	Login with valid/invalid credentials; marks entry with 0, 50, 100, 101
Input Validation	Verify each input field rejects invalid data	Text in marks field = error message
Integration Testing	Test interactions between modules	Student registers -- enrolls in course -- marks captured -- report reflects all
Performance Testing	Measure response times for critical operations	Report generation time for 50 students with 5 courses each
Usability Testing	Evaluate navigation efficiency	Complete student registration in ≤ 3 clicks from dashboard
Security Testing	Test authentication and session management	Invalid login attempts = account lockout; direct URL access without login
Reliability Testing	Repeated operations to ensure consistency	Add/remove course enrolment 20 times; verify no duplication

Question 1.3

Hardware: Intel Core i5 / 8GB RAM / 256GB SSD / Windows 11

Software: Visual Studio 2026, .NET Framework 4.7.2 / .NET 6.0

Browser: Windows Forms Application

Testing Tools: Visual Studio built-in debugger, GitHub, Snipping Tool (screenshots), Notepad++

Test Data Sets:

- Valid users: admin/1234
- Invalid users: test/user, "/"
- Marks: 0-100, -5, 101, "ABC"
- Courses: PROG101, etc

Task 2

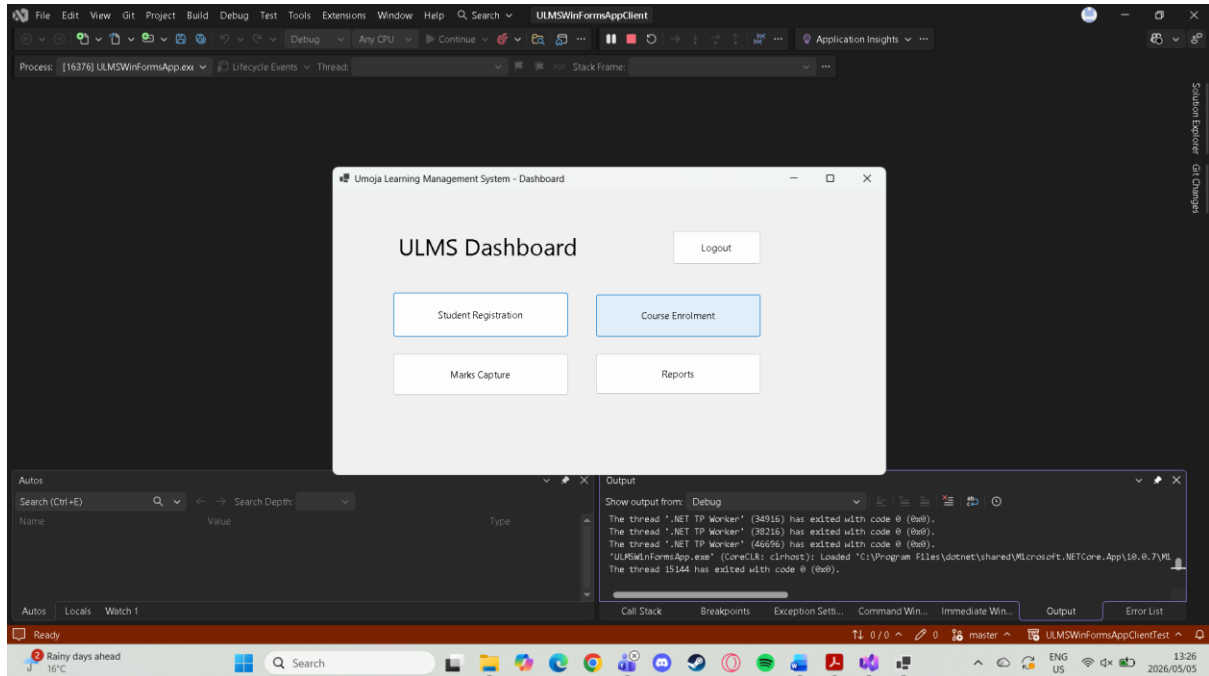
TC ID	Module/ Feature	Input Data	Expected Result	Actual Result	Status
Tc-1	Login-Valid credentials	Username: admin Password: 1234	Dashboard loads successfully ; welcome message displays	Dashboard loaded successfully	Pass
Tc-2	Login-Invalid password	Username: admin Password: wrong	Error: "Invalid login credentials"	Login succeeds incorrectly due to faulty validation	Fail
Tc-3	Login – Invalid username	Username: wrong Password: 1234	Error: "Invalid login credentials"	Login succeeds incorrectly due to faulty validation	Fail
Tc-4	Course Enrolment – First enrolment (valid)	Student ID: STU001, Name: John Doe Course: PROG101	Success message; enrolment record added	"Enrolment completed successfully!" message appears	Pass
Tc-5	Course Enrolment – Duplicate course (same student, same course)	Student ID: STU001, same student Course: PROG101 (same course)	Error message: "Already enrolled in this course"	Duplicate enrolment added successfully	Fail
Tc-6	Marks Capture – Valid numeric marks (All 75)	Student ID: STU001 Subject1: 75, Subject2: 75, Subject3: 75	Marks saved; average calculated correctly = 75.00	Marks saved; incorrect average = 175.00 (due to $75+75+75/3$ formula)	Fail
TC - 7	Marks Capture – Valid numeric marks (Pass boundary: 50 exactly)	Student ID: STU001 Subjects: 50, 50, 50	Marks saved; average = 50.00; Final Status = PASS	Marks saved; incorrect average = 116.67; Status = PASS	Fail
Tc-8	Marks Capture –	Student ID: STU001 Subjects: 50, 50, 47	Marks saved;	Marks saved; incorrect	Fail

	Valid numeric marks (Fail: Average 49)		average = 49.00; Final Status = FAIL	average calculation; Status incorrect	
Tc-9	Marks Capture – Non-numeric input	Student ID: STU001 Subject1: ABC, Subject2: 75, Subject3: 75	Error: "Please enter numeric values"	Application crashes with unhandled FormatException	Fail
Tc-10	Marks Capture – Empty input fields	Student ID: STU001 Subject fields left blank	Validation error: "All fields required"	Application crashes with FormatException on empty string	Fail
TC - 11	Student Registration – Valid new student	Student Number: S12345 Full Name: Mary Jane Email: mary@umojagroup.com Age: 22 Programme: Software Development	Success message; student added to list	Registration succeeds	Pass
TC - 12	Student Registration – Duplicate student number	Student Number: S12345 (same as TC-011) Full Name: Duplicate	Error: "Student number already exists"	Duplicate added without error	Fail

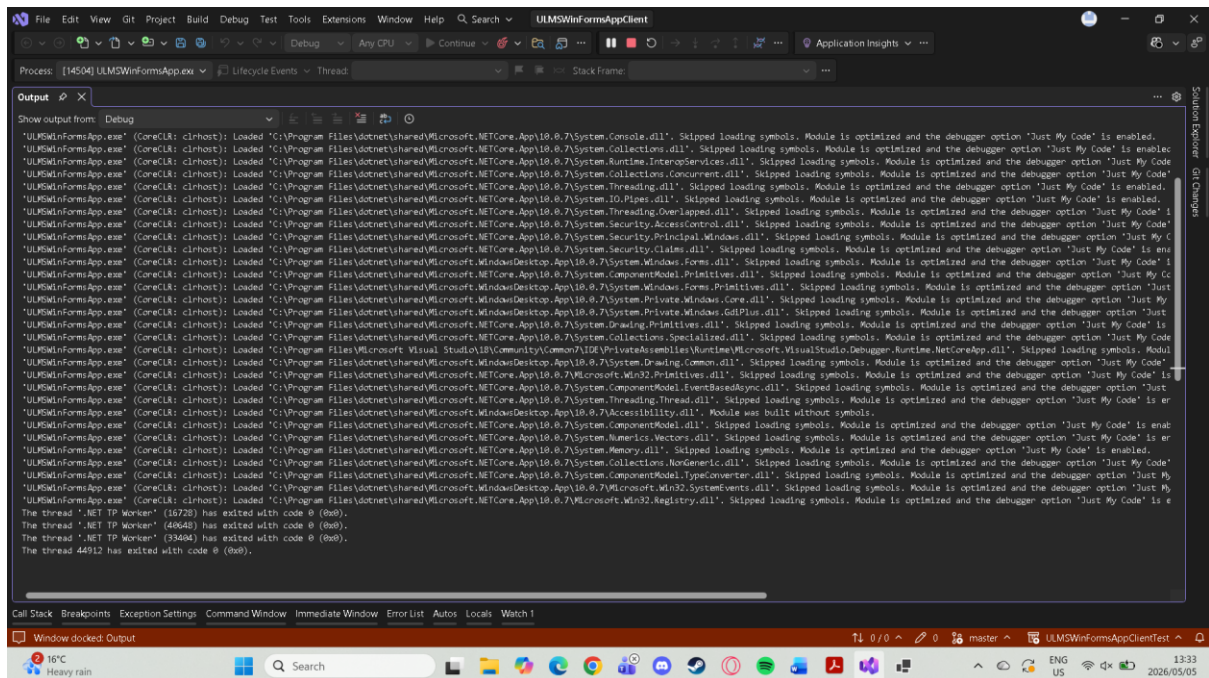
Task 3

Test Case 1:

Screenshot Result:



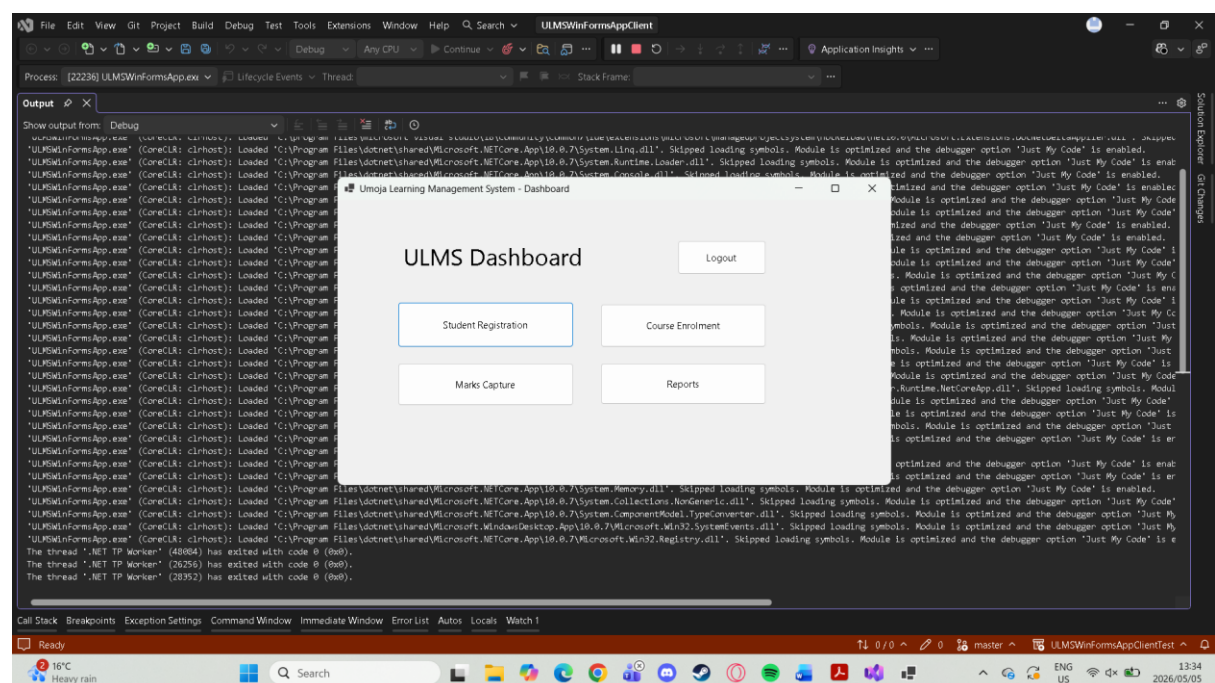
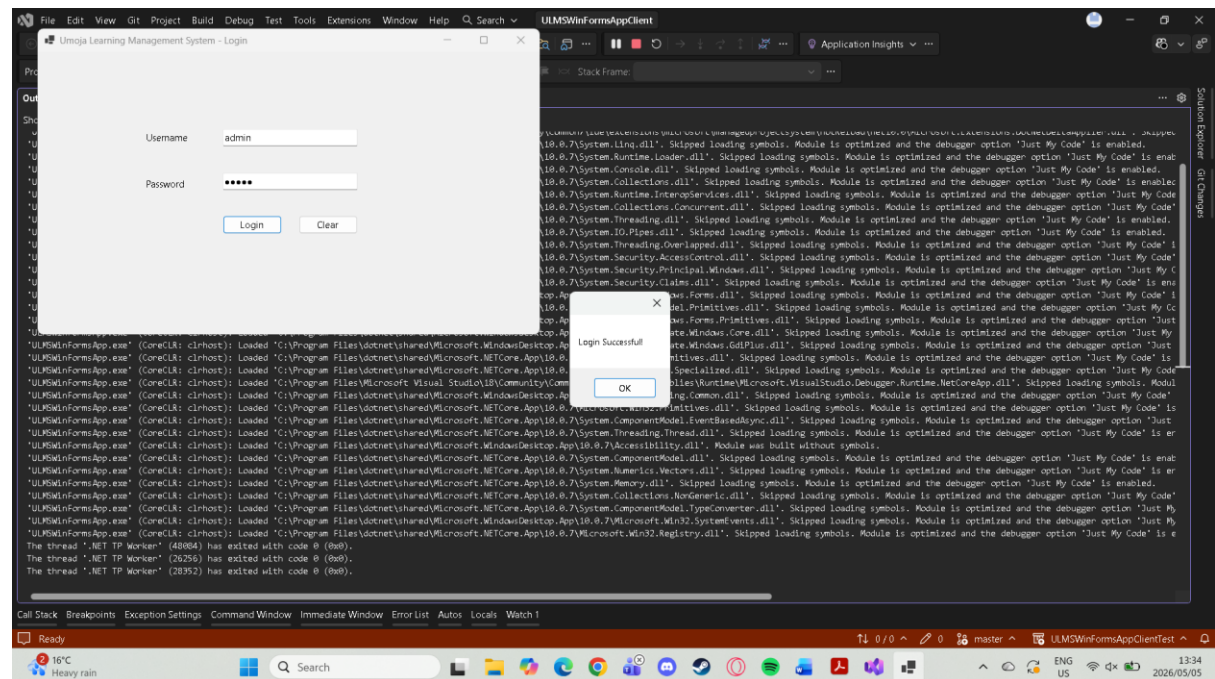
Execution Logs:



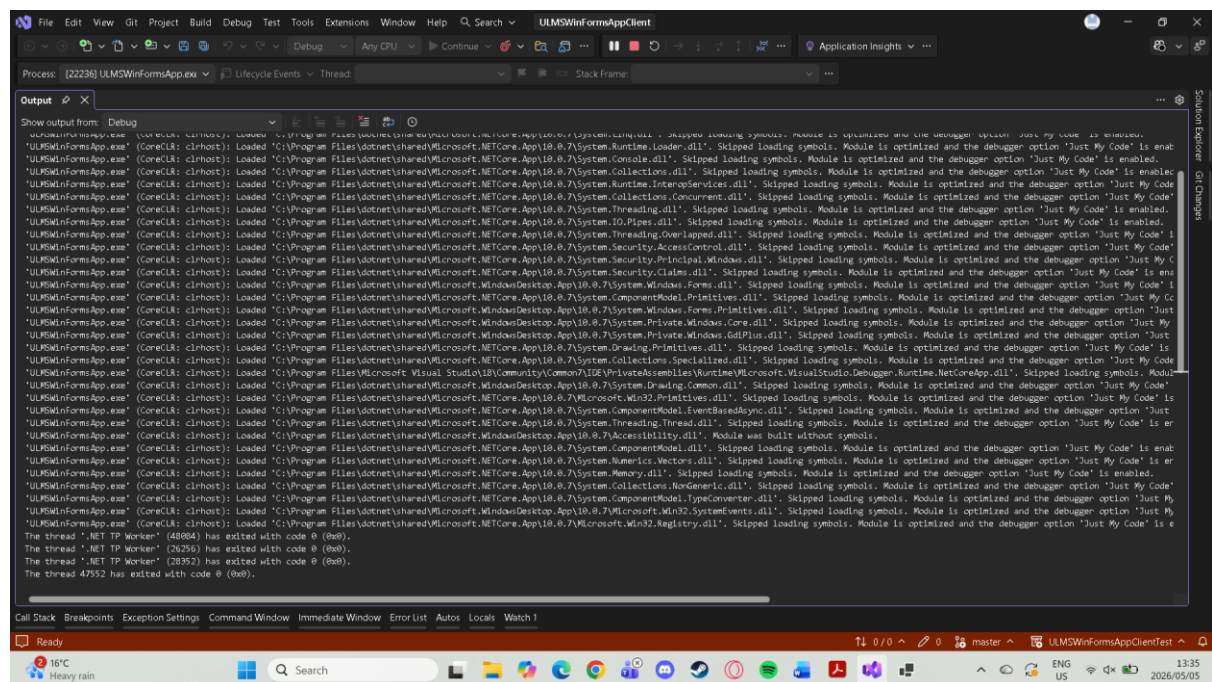
Test Case 2:

(This looks the same as the first test case but I had typed 'wrong' into the password text block)

Screenshot Result:

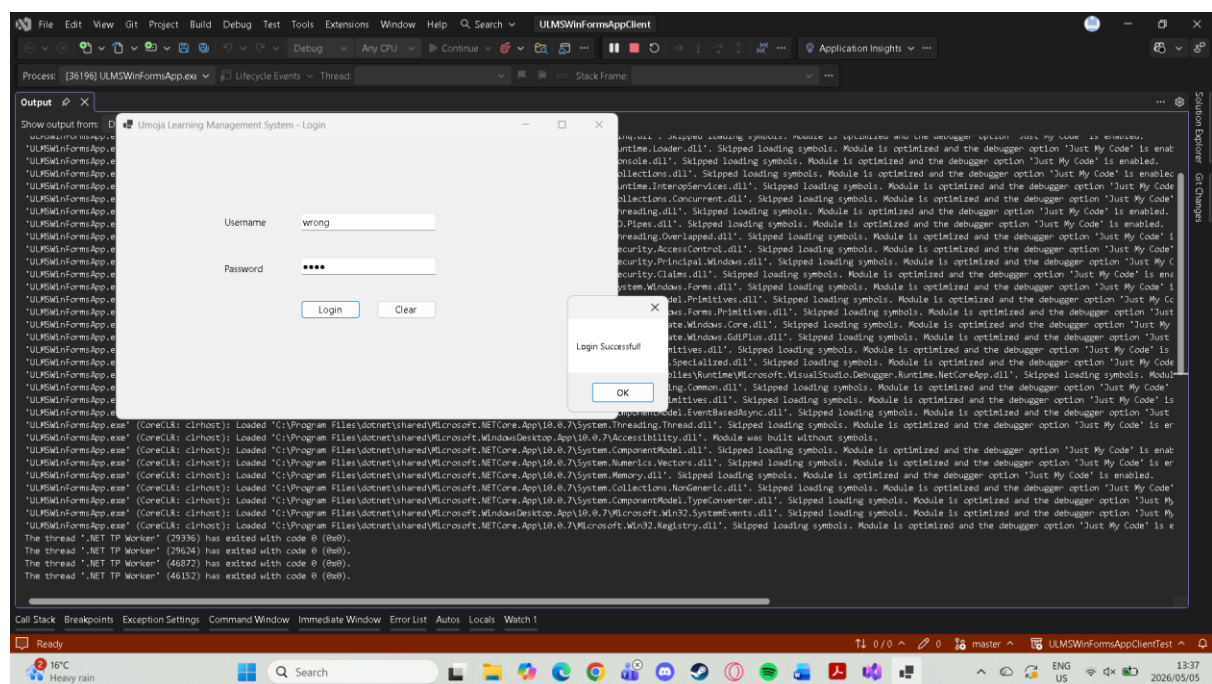


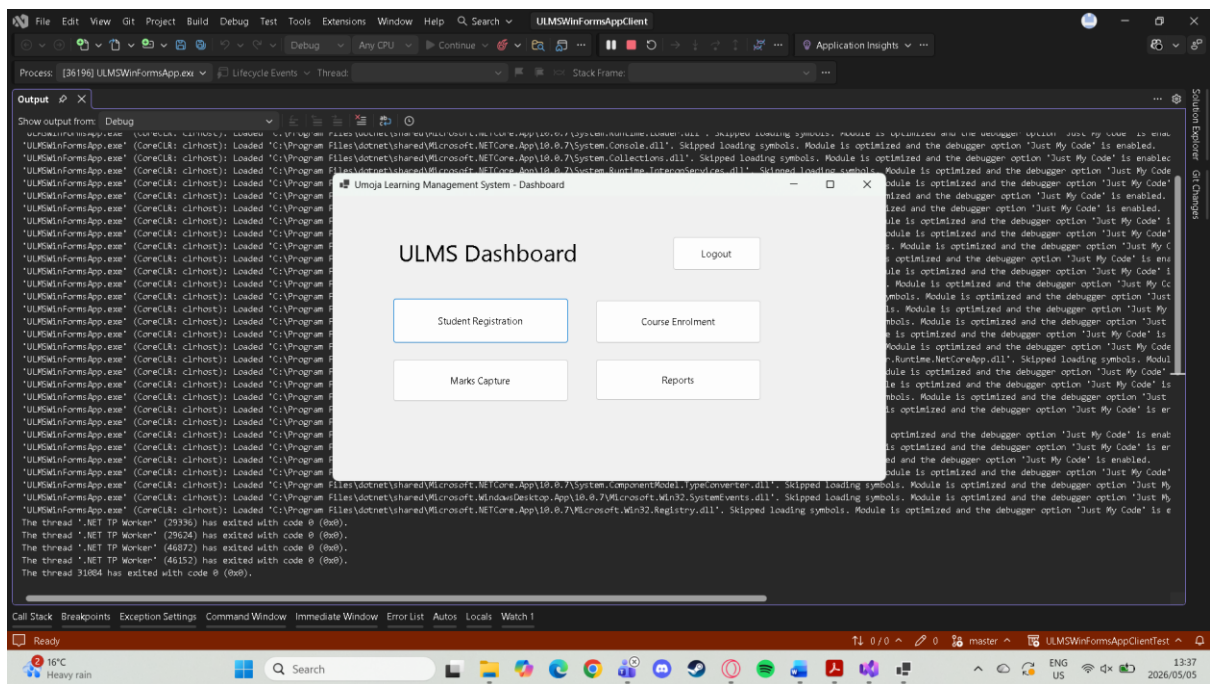
Execution Logs:



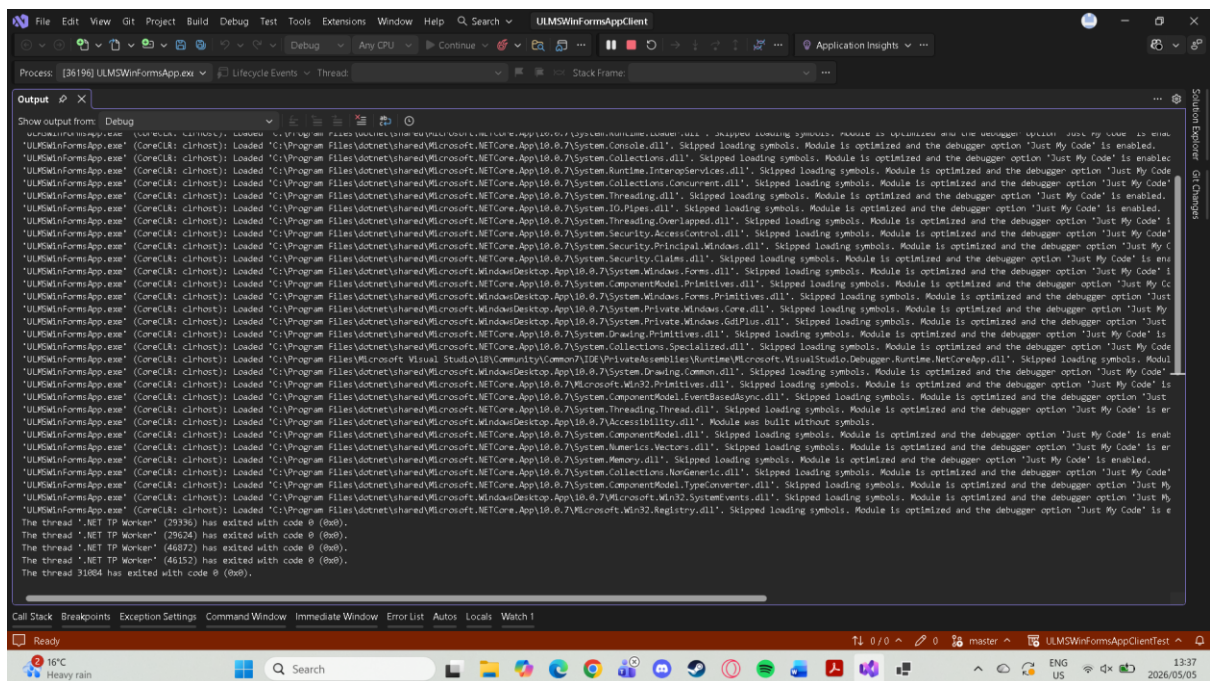
Test Case 3:

Screenshot Result:



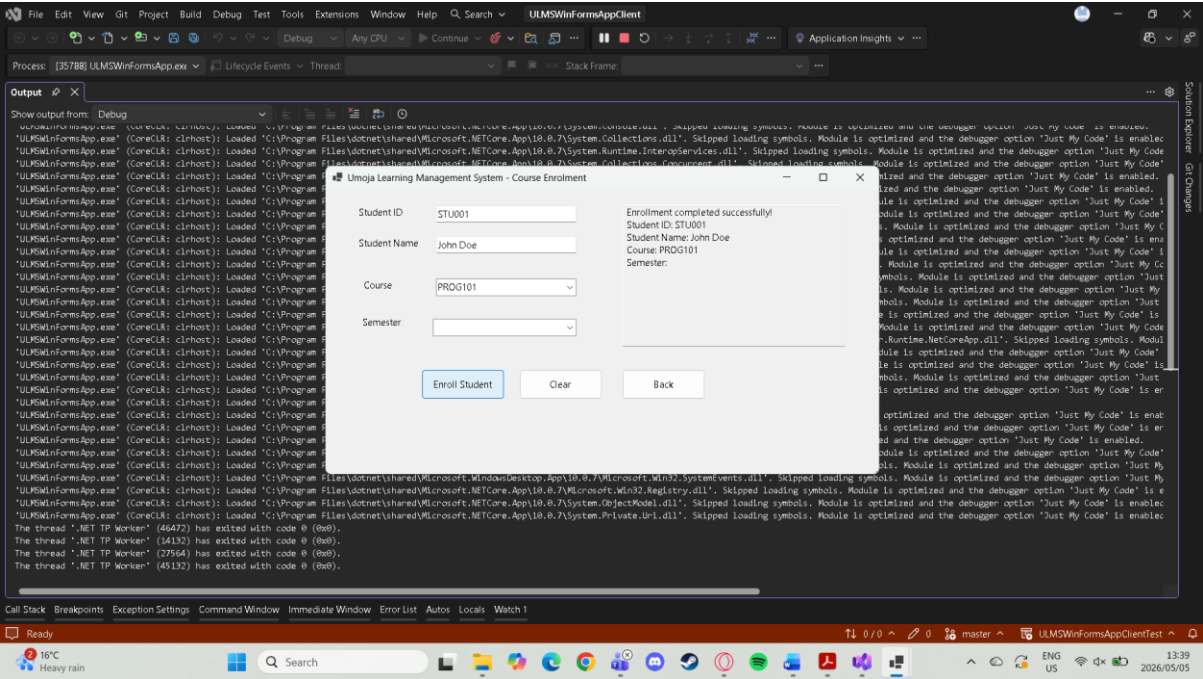


Execution Logs:

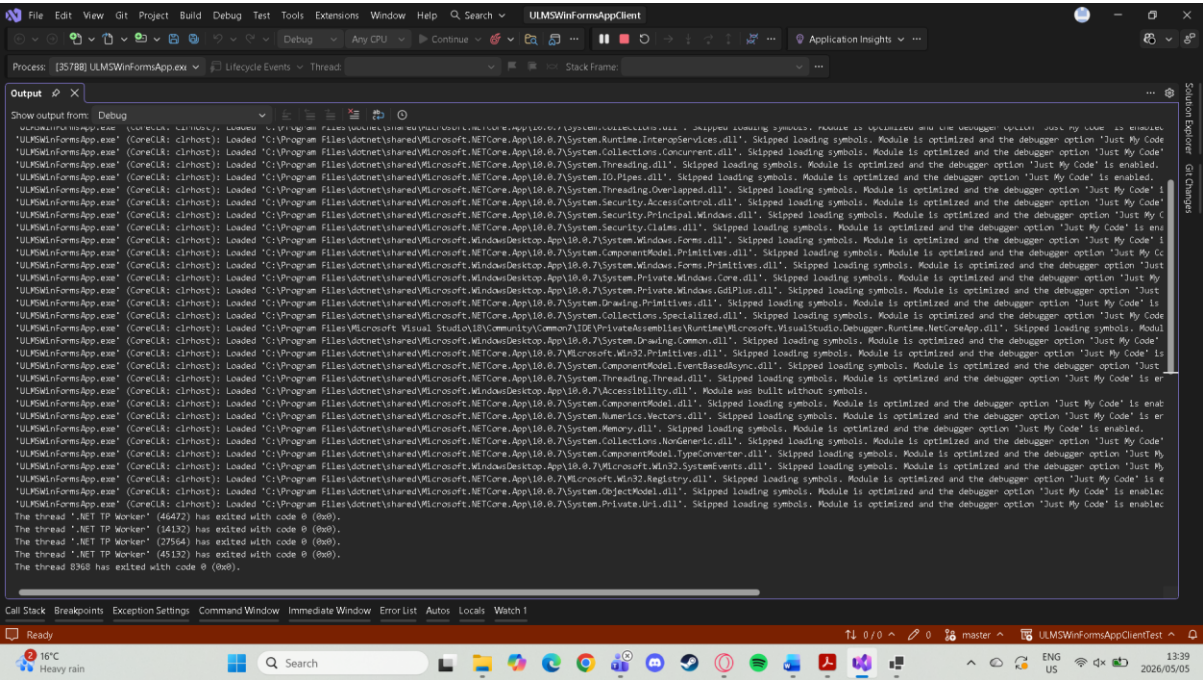


Test Case 4:

Screenshot Result:

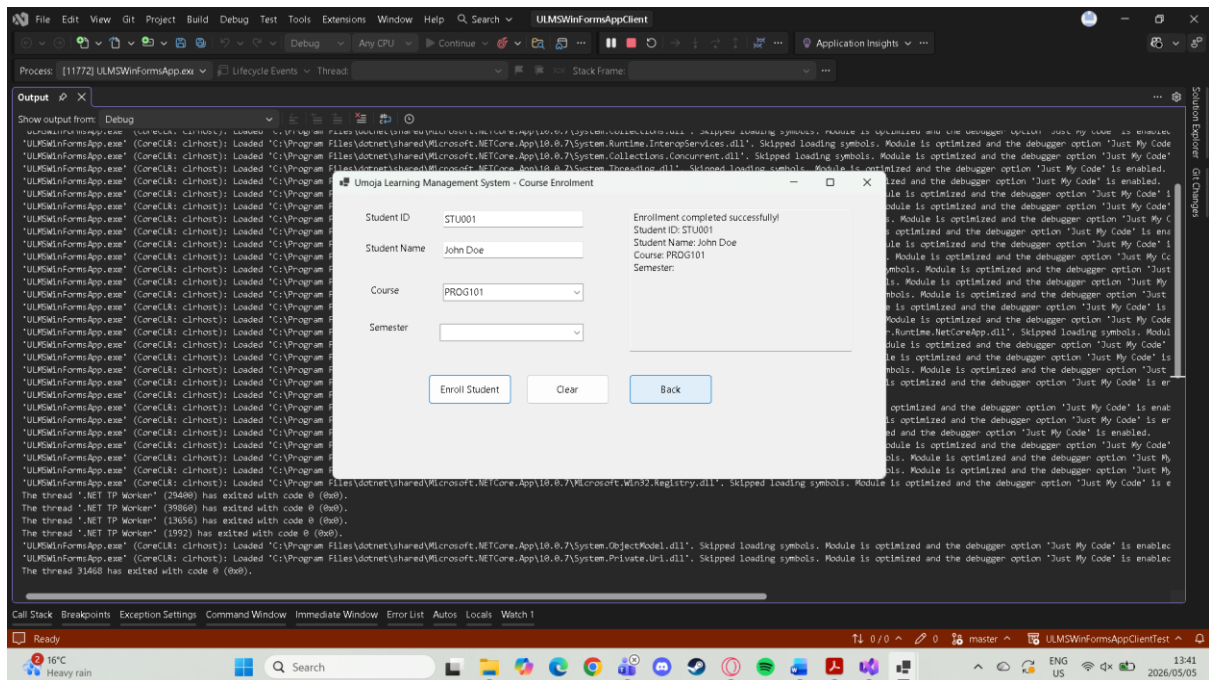


Execution Logs:

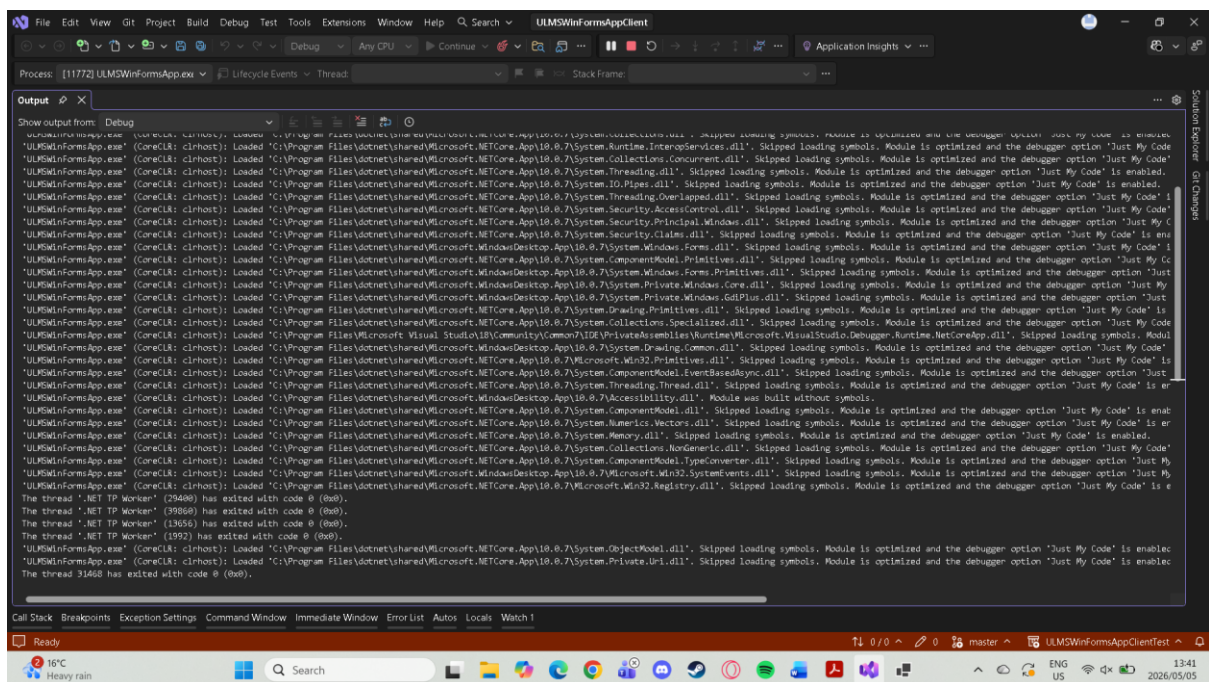


Test Case 5:

Screenshot Result:

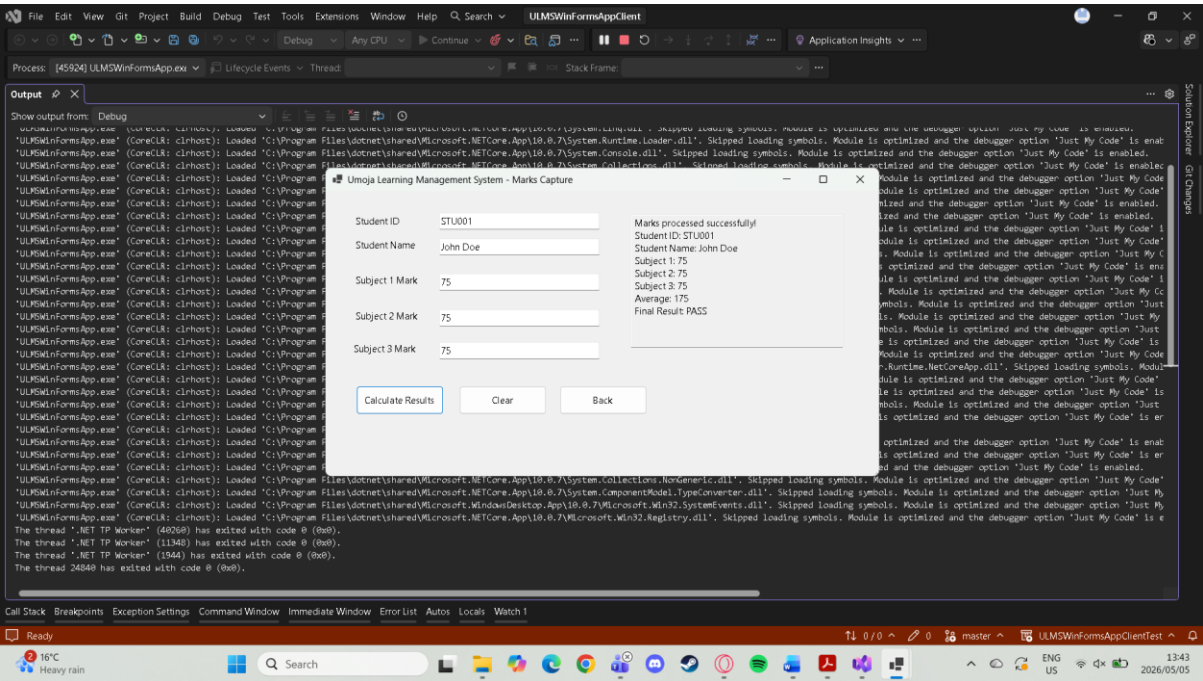


Execution Logs:

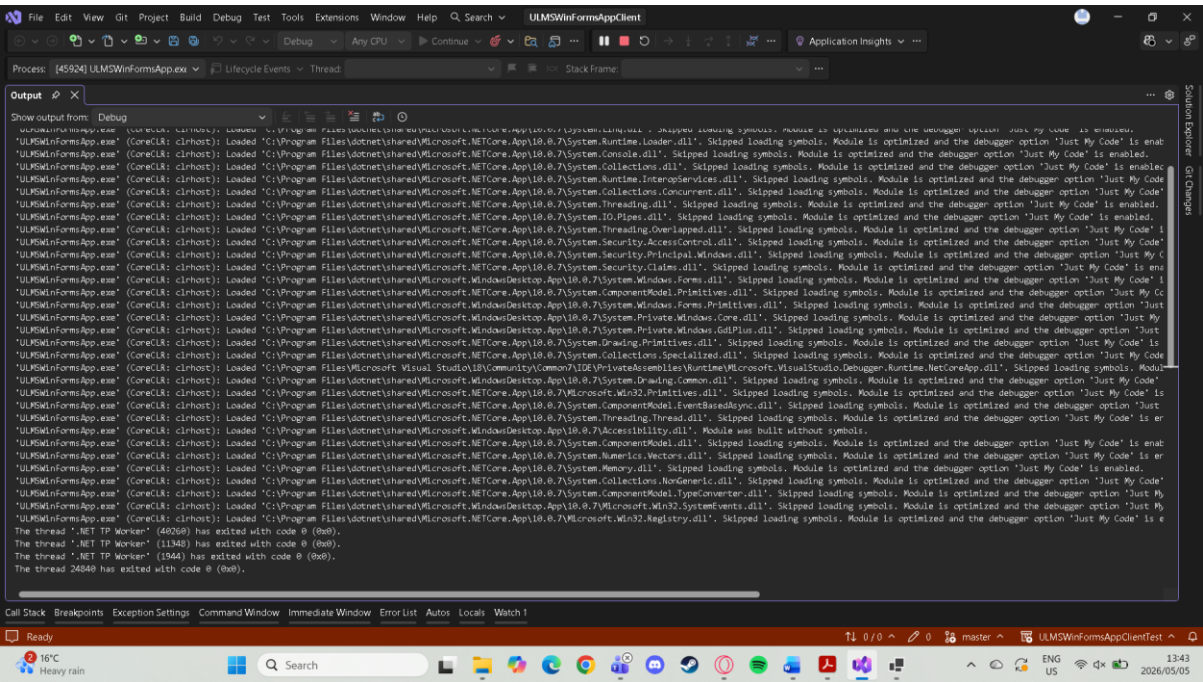


Test Case 6:

Screenshot Result:

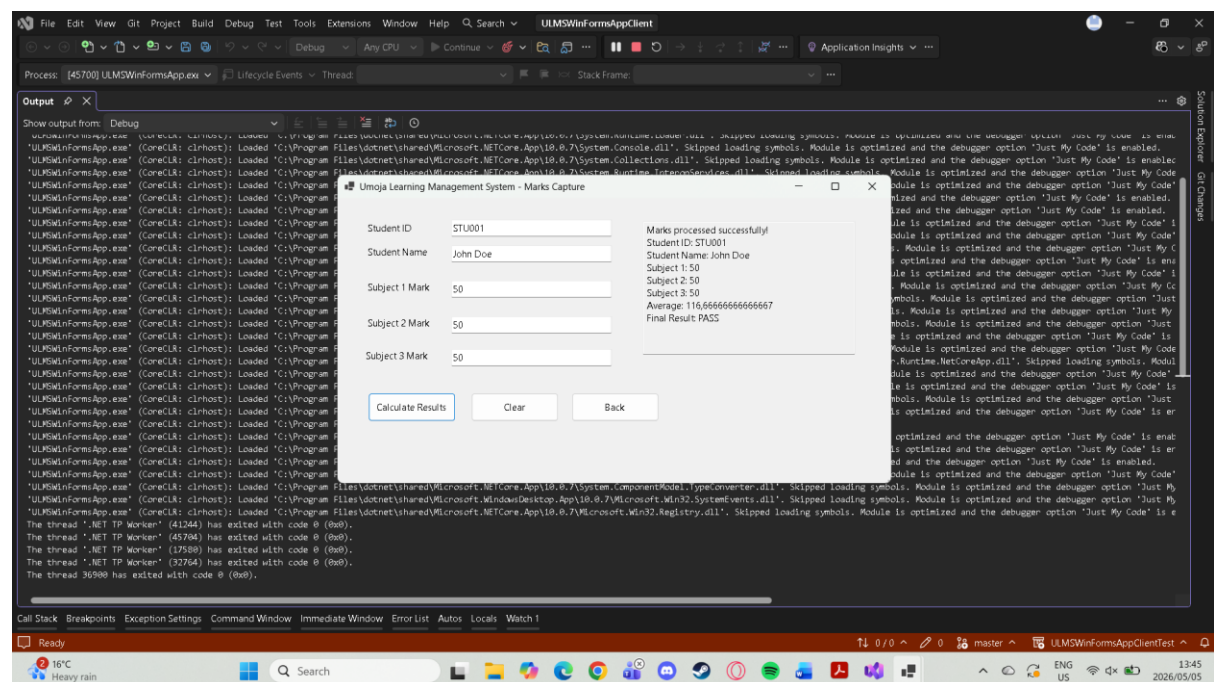


Execution Logs:

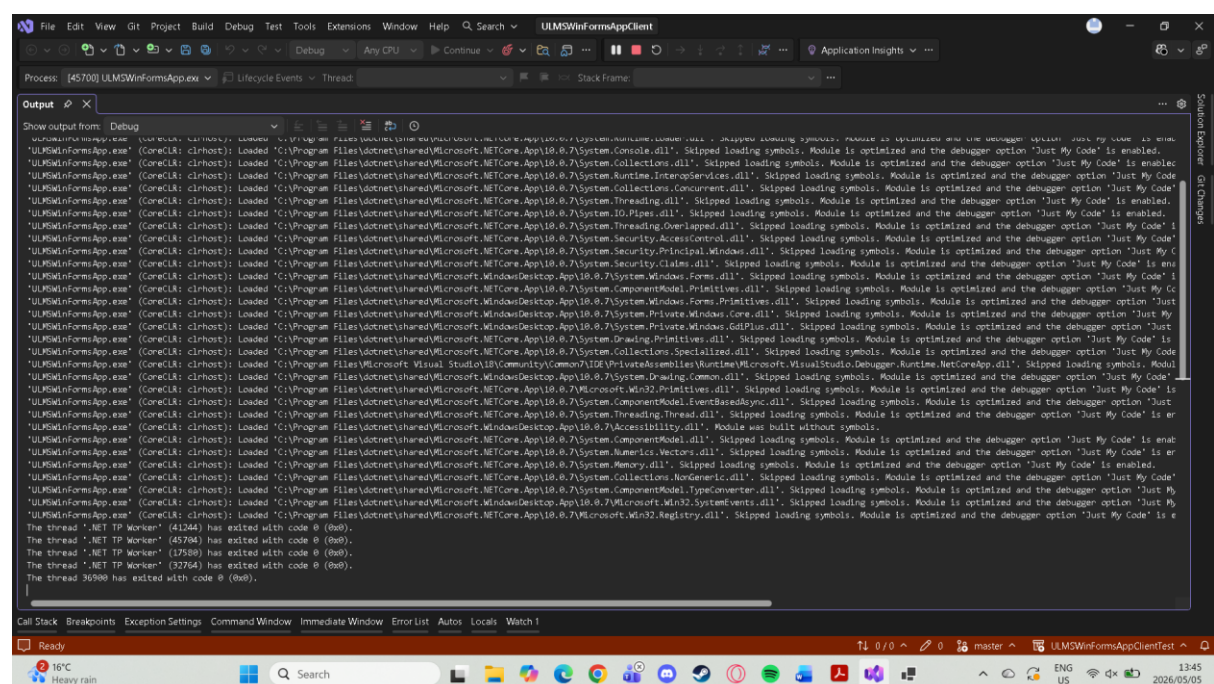


Test Case 7:

Screenshot Result:

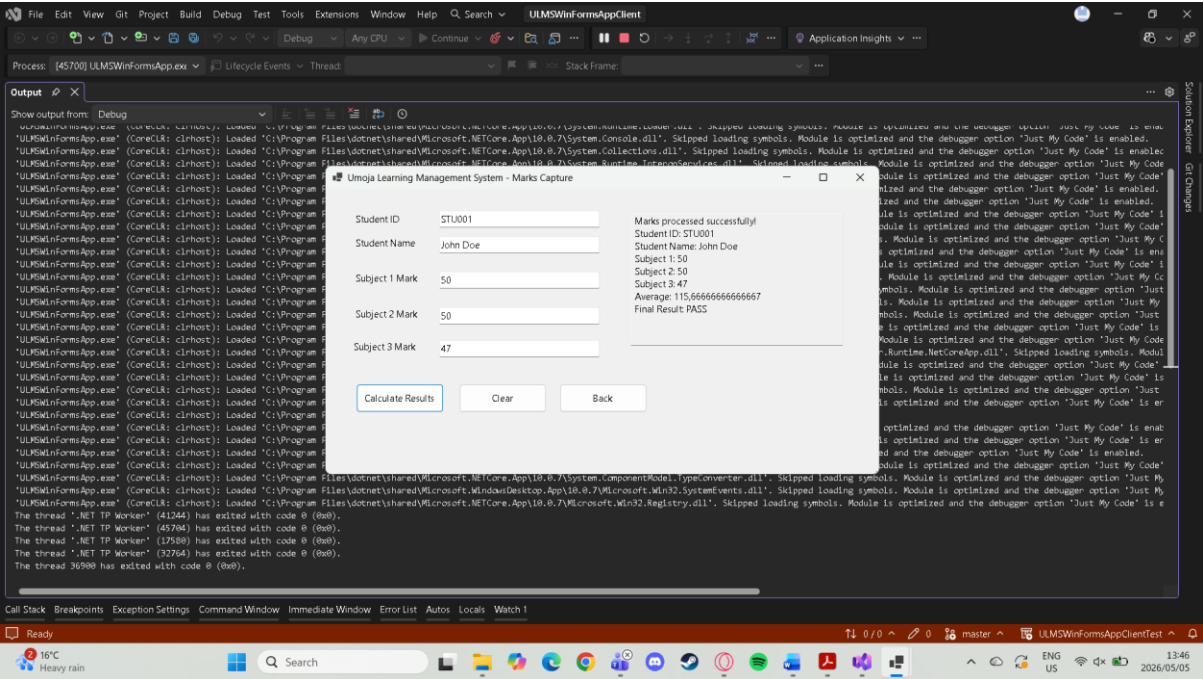


Execution Logs:

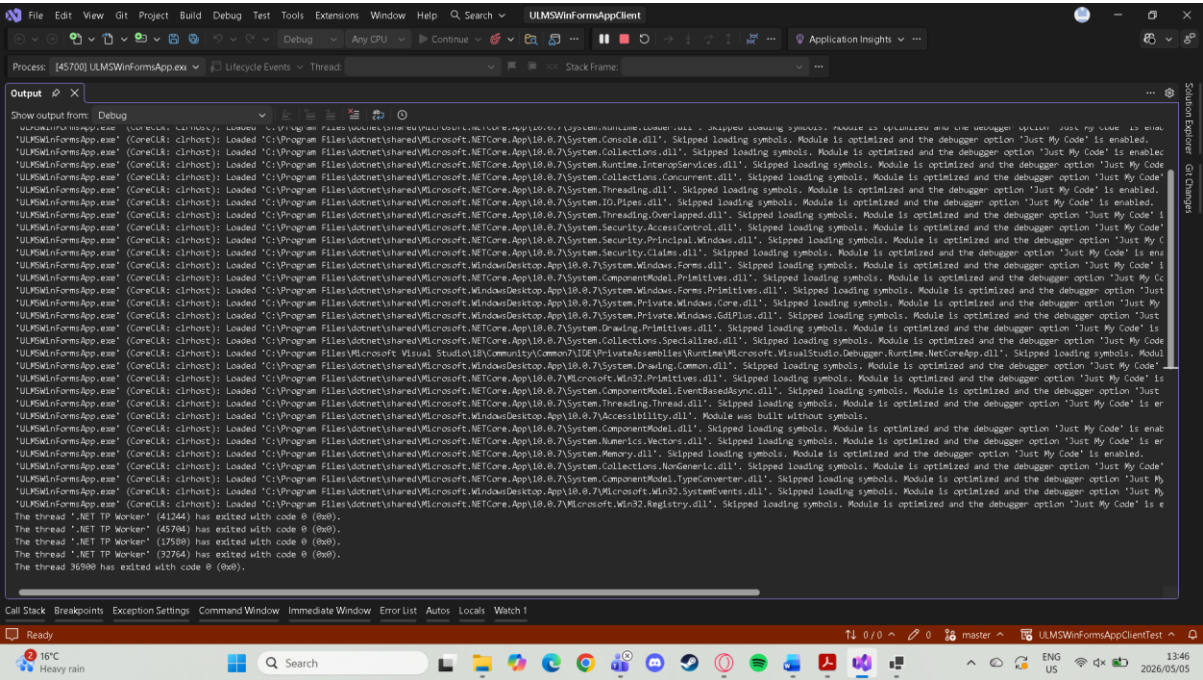


Test Case 8:

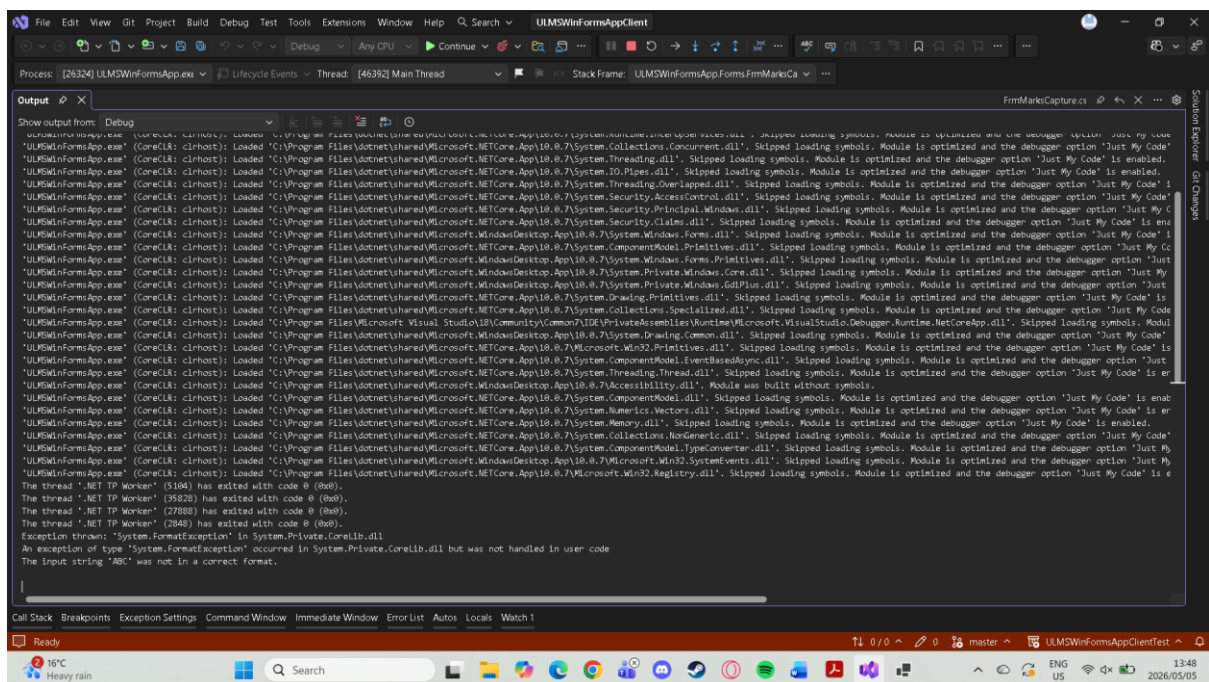
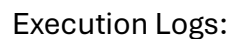
Screenshot Result:



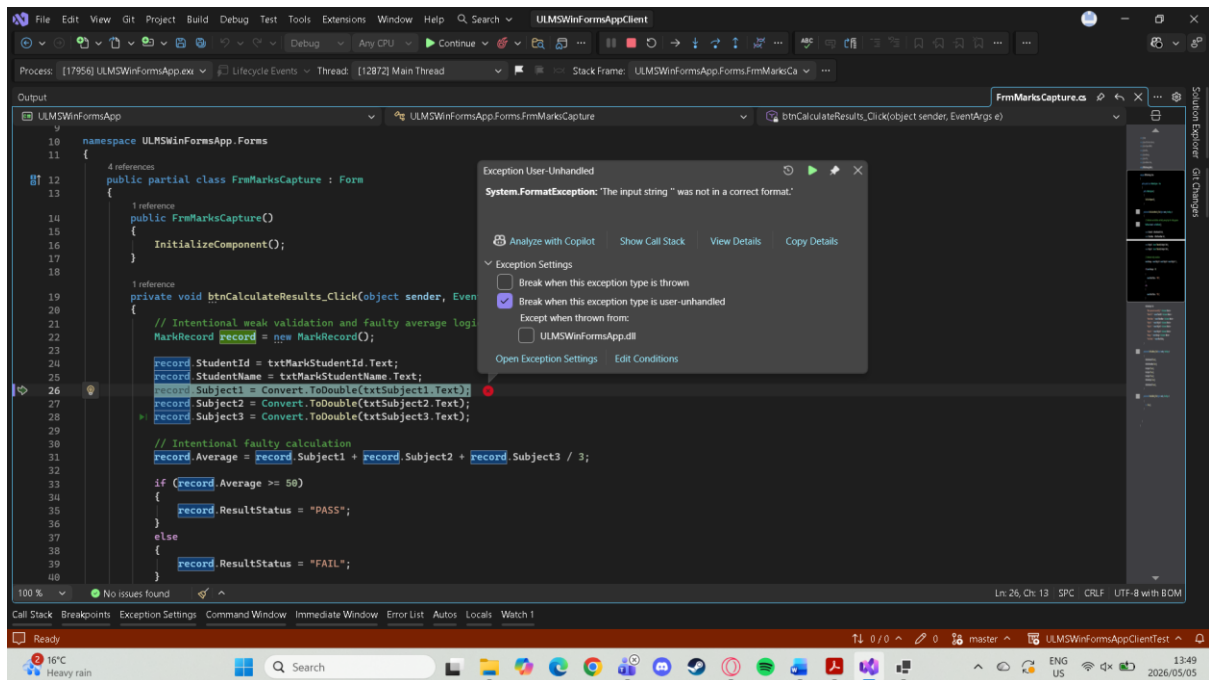
Execution Logs:



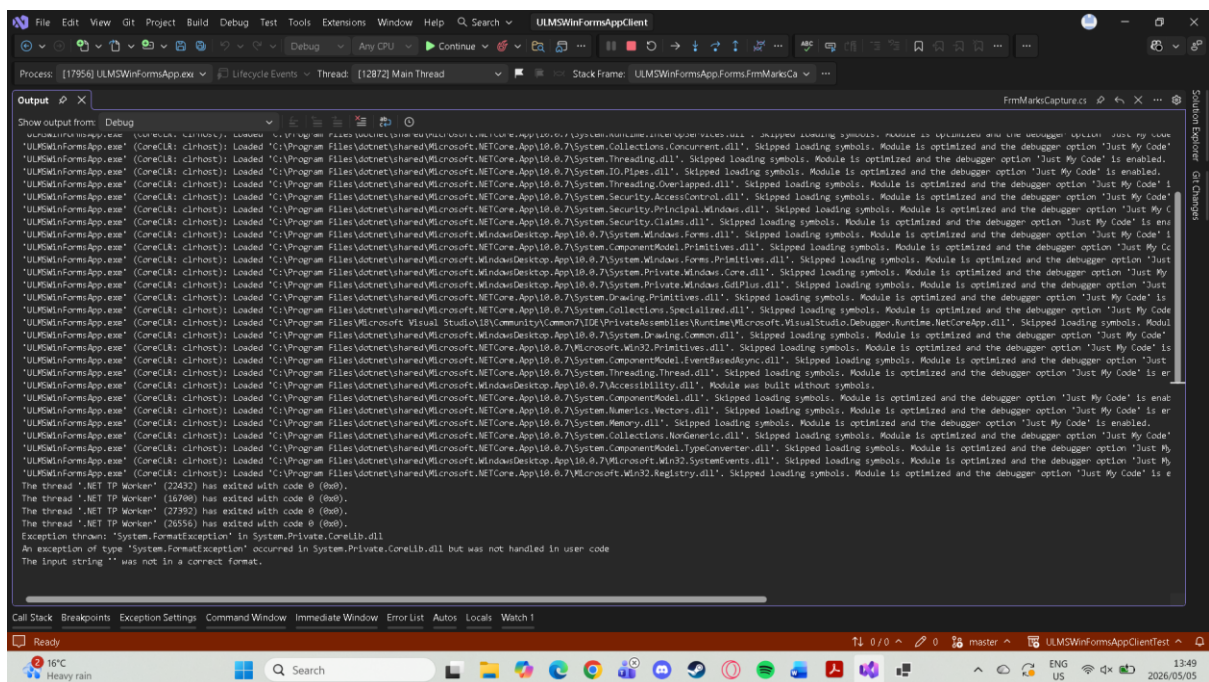
Screenshot Result:



Screenshot Result:

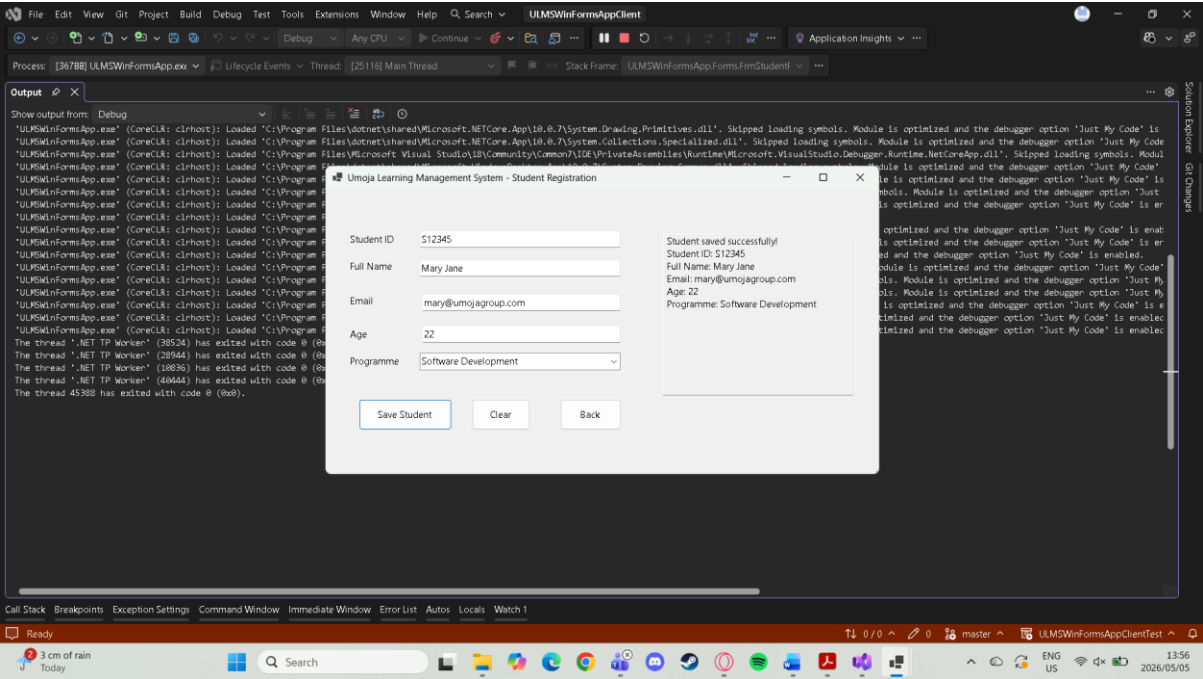


Execution Logs:

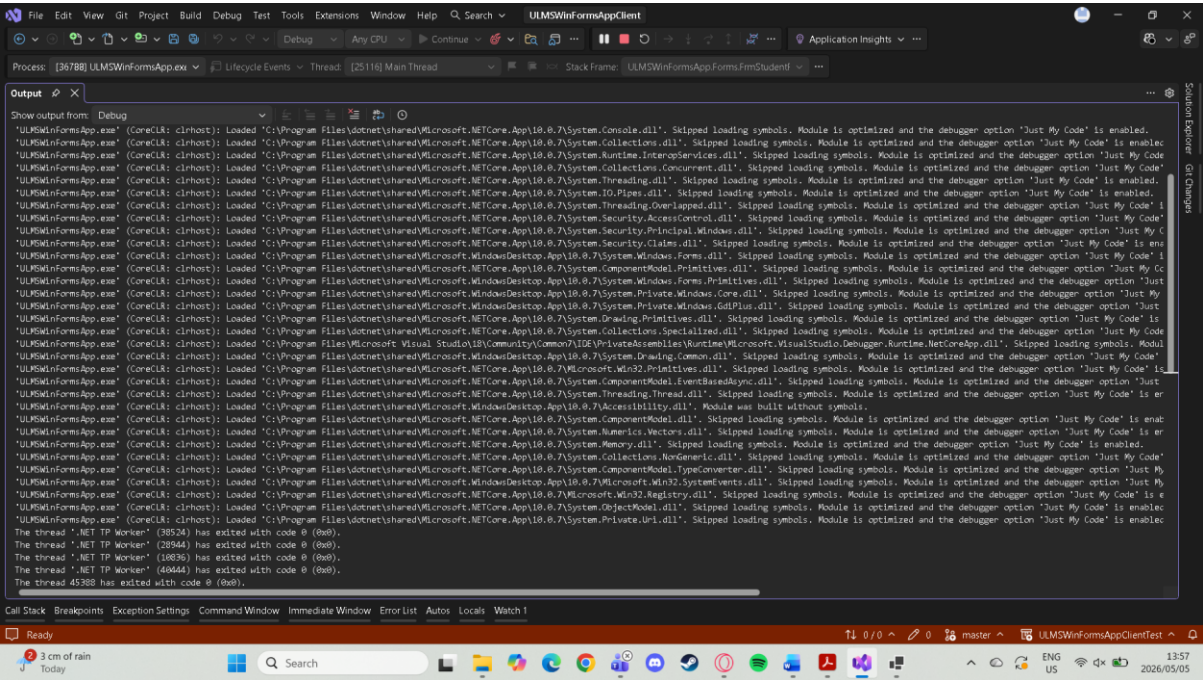


Test Case 11:

Screenshot Result:

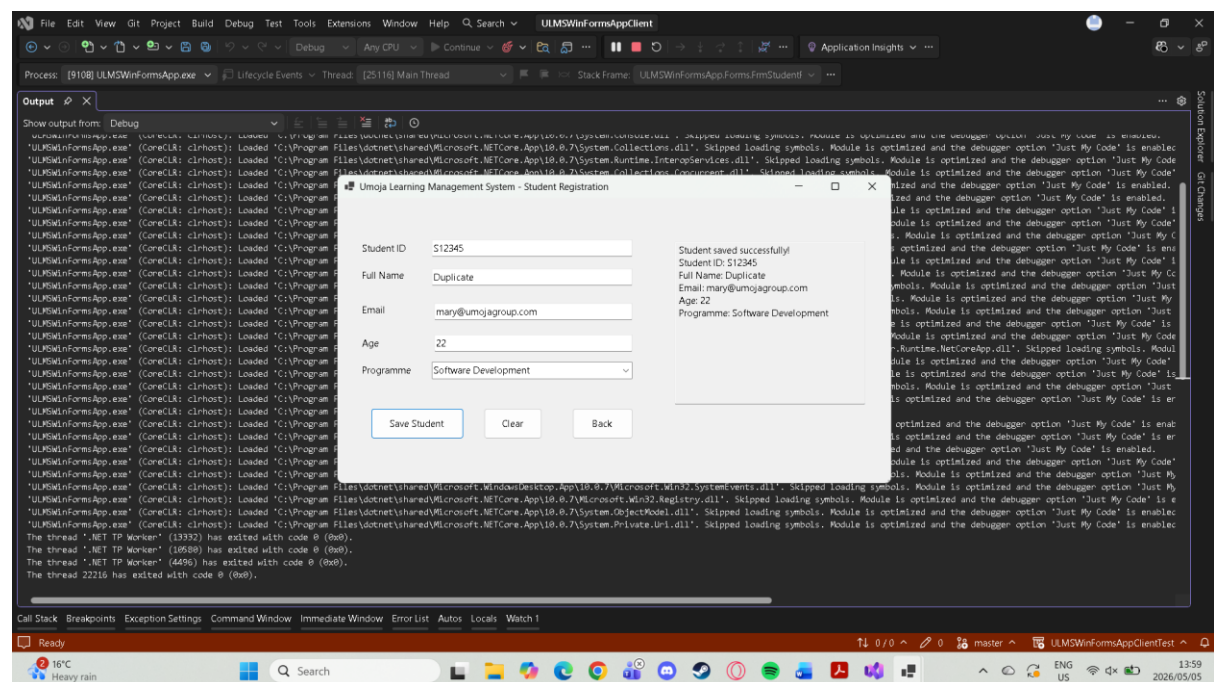


Execution Logs:

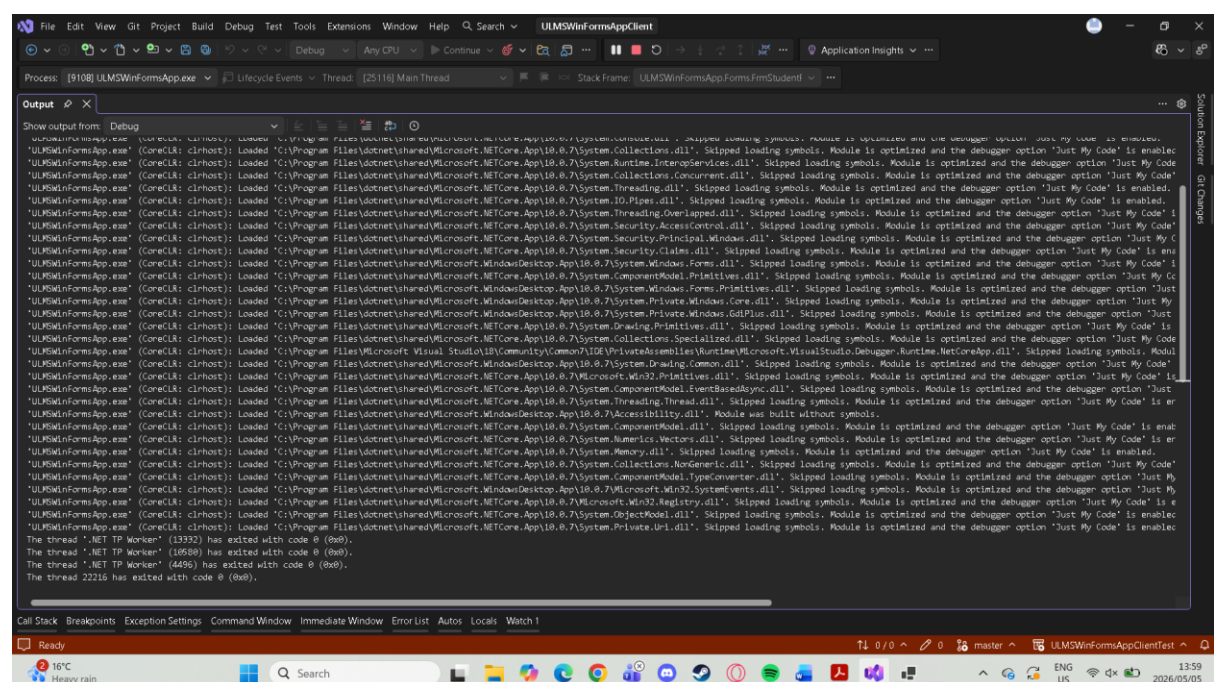


Test Case 12:

Screenshot Result:



Execution Logs:



Task 4

Performance Testing:

Method: Measured time from click until report appears in txtReportOutput. Used Stopwatch

Report Type	Observed Load Time (ms)	Threshold	Status
Student Summary	4000+	≤2000	Fail
Marks Report	4000+	≤2000	Fail
Enrolment Report	4000+	≤2000	Fail

Interpretation: The code uses Thread.Sleep(4000), a hard-coded 4-second delay, on the UI thread, freezing the application. This is not real performance load but an intentional bottleneck

Usability Testing:

Form	Usability Issue
Login Form	No feedback for empty fields; "Clear" button not wired to click event (btnClear has no Click handler in .cs file)
Student Registration Form	Age field accepts any text which causes crash on int.Parse; no duplicate check
Course Enrolment Form	No validation of required fields; can enrol with blank Student ID/Name
Marks Capture Form	No input validation; crash on non-numeric or empty marks; incorrect average formula
Reports Form	Report type selection required but no validation if none selected

Interpretation: All forms have "Back" buttons that simply close the form. Which could be acceptable, but there is no logout from dashboard. Dashboard form logout opens new login but does not close dashboard properly (it calls this.Close() after login.Show(); login may appear briefly then dashboard closes).

Security Testing:

Attack Vector	Actual Behavior	Severity
Faulty logic ()	Any username with password "1234" logs in; OR any password with username "admin" logs in	Critical
No empty field check	Empty username/password = no message (condition false, goes to else "Invalid login credentials")	Medium
Hardcoded credentials	Credentials embedded in code	High
Direct dashboard access	After login, dashboard shown, but Program.cs starts with the Login Form – no direct bypass unless user closes login. Actually, login must be successful to reach dashboard. However, faulty logic still critical.	Critical

Interpretation: Authentication is severely broken. The || should be &&. No session token, but dashboard cannot be accessed without login because Application.Run(new FrmLogin()) is the entry point. Still, the login condition is wrong

Reliability Testing:

Operation	Failure	Root Cause
Student Registration with Age non-numeric	Crash (FormatException)	int.Parse(txtAge.Text) without TryParse
Marks Capture with non-numeric marks	Crash (FormatException)	Convert.ToDouble(...) on empty/non-numeric text
Marks Capture with valid numbers	Wrong average	Subject1 + Subject2 + Subject3 / 3 (operator precedence)
Enrolment – duplicate enrolment	No error – allows duplicate	No check for existing enrolment; just prints to textbox, no storage

Report generation without selecting type	Shows "No report type selected." – acceptable, but no validation	No if (string.IsNullOrEmpty(reportType)) check – default empty string triggers else clause. Works but not user-friendly.
--	--	--

Interpretation: Reliability is poor due to unhandled exceptions. Application crashes on any invalid input in age or marks fields.

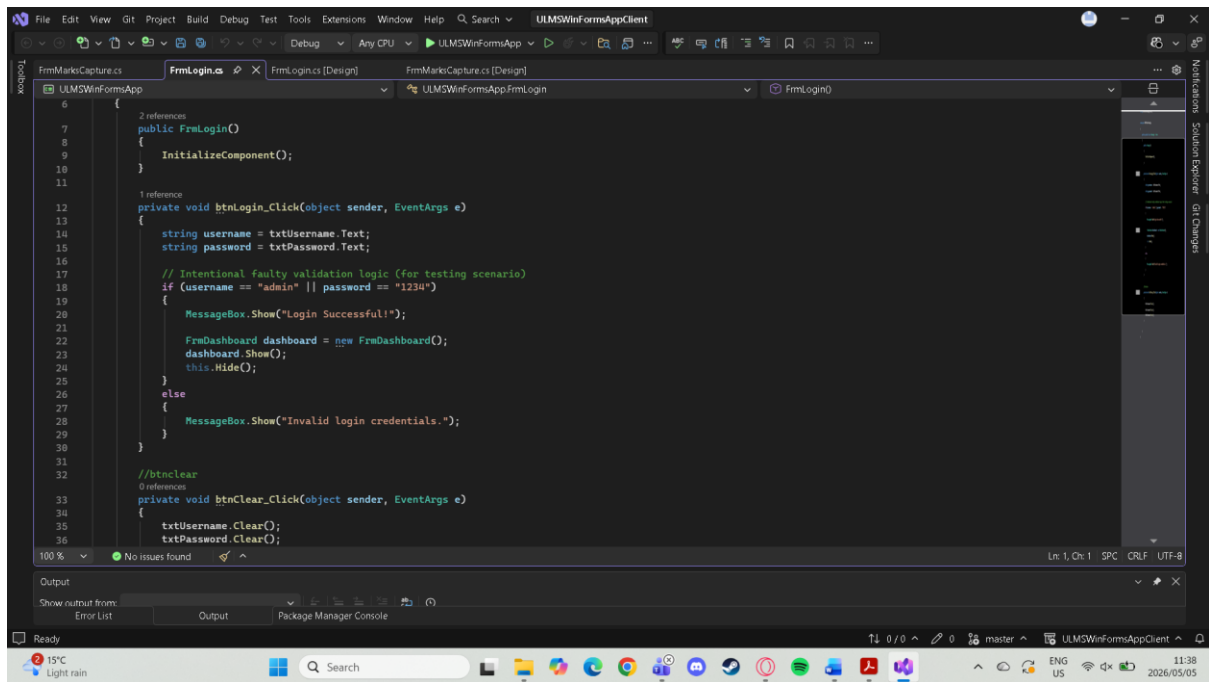
Task 5

Defect 1

Location: Login Page

Issue: Faulty Logic

Original Faulty Code:

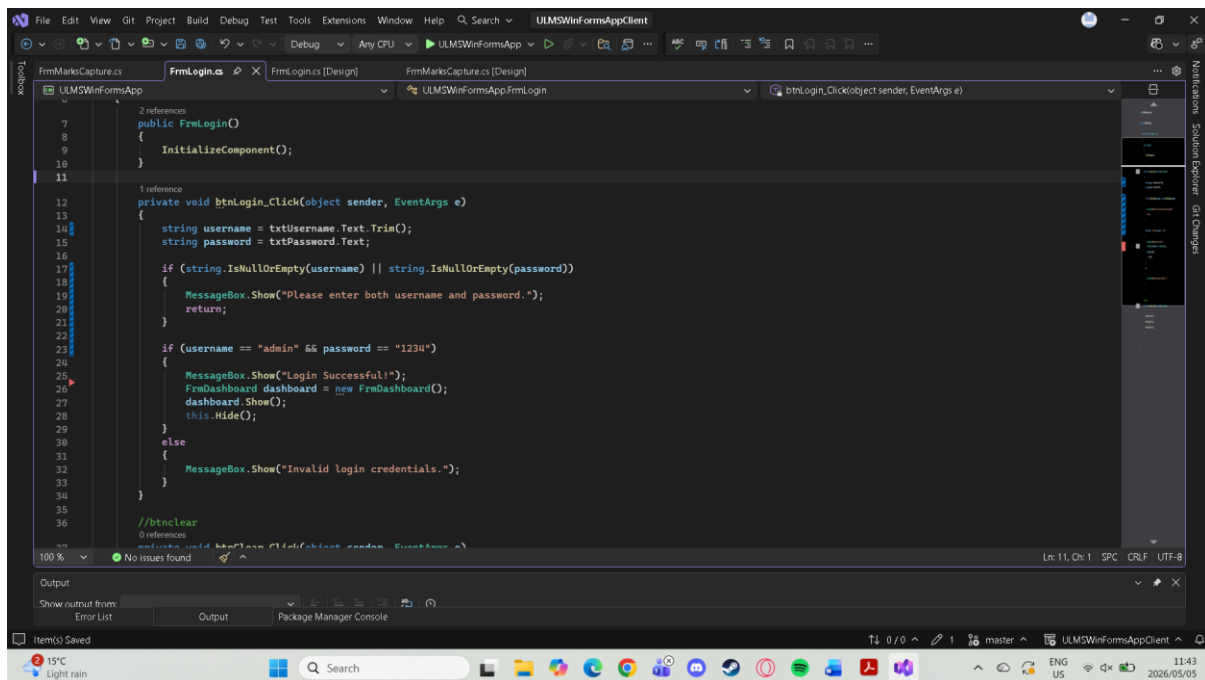
A screenshot of the Visual Studio IDE showing a C# code file named 'FrmLogin.cs'. The code is for a Windows Forms application. It includes a 'public FrmLogin()' method that calls 'InitializeComponent()'. Below this is a 'private void btnLogin_Click(object sender, EventArgs e)' method. Inside this method, it gets 'username' and 'password' from text boxes. It then has a conditional statement: 'if (username == "admin" || password == "1234")'. This is the faulty logic. If true, it shows a 'Login Successful!' message box and navigates to a 'FrmDashboard'. Otherwise, it shows an 'Invalid login credentials.' message box. There is also a 'btnClear' section that clears the text boxes. The status bar at the bottom indicates 'No issues found'. The Windows taskbar at the very bottom shows the date as 2026/05/05 and time as 11:38.

```
6 {
7     2 references
8     public FrmLogin()
9     {
10         InitializeComponent();
11     }
12
13     1 reference
14     private void btnLogin_Click(object sender, EventArgs e)
15     {
16         string username = txtUsername.Text;
17         string password = txtPassword.Text;
18
19         // Intentional faulty validation logic (for testing scenario)
20         if (username == "admin" || password == "1234")
21         {
22             MessageBox.Show("Login Successful!");
23
24             FrmDashboard dashboard = new FrmDashboard();
25             dashboard.Show();
26             this.Hide();
27         }
28         else
29         {
30             MessageBox.Show("Invalid login credentials.");
31         }
32     }
33
34     //btnClear
35     0 references
36     private void btnClear_Click(object sender, EventArgs e)
37     {
38         txtUsername.Clear();
39         txtPassword.Clear();
40     }
41 }
```

Issue Description: The condition uses || (OR) instead of && (AND). This allows login with any username if password is "1234", or with any password if username is "admin"

Why It occurs: Developer mistakenly used OR instead of AND.

Fixed Code:



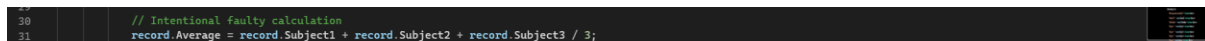
```
7 public FrmLogin()
8 {
9     InitializeComponent();
10 }
11
12 1 reference
13 private void btnLogin_Click(object sender, EventArgs e)
14 {
15     string username = txtUsername.Text.Trim();
16     string password = txtPassword.Text;
17
18     if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
19     {
20         MessageBox.Show("Please enter both username and password.");
21         return;
22     }
23
24     if (username == "admin" && password == "1234")
25     {
26         MessageBox.Show("Login Successful!");
27         FrmDashboard dashboard = new FrmDashboard();
28         dashboard.Show();
29         this.Hide();
30     }
31     else
32     {
33         MessageBox.Show("Invalid login credentials.");
34     }
35 }
36
37 //btnClear
38 0 references
39 private void btnClear_Click(object sender, EventArgs e)
```

Defect 2

Location: MarksCapture

Issue: Incorrect Average Calculation

Original Faulty Code:

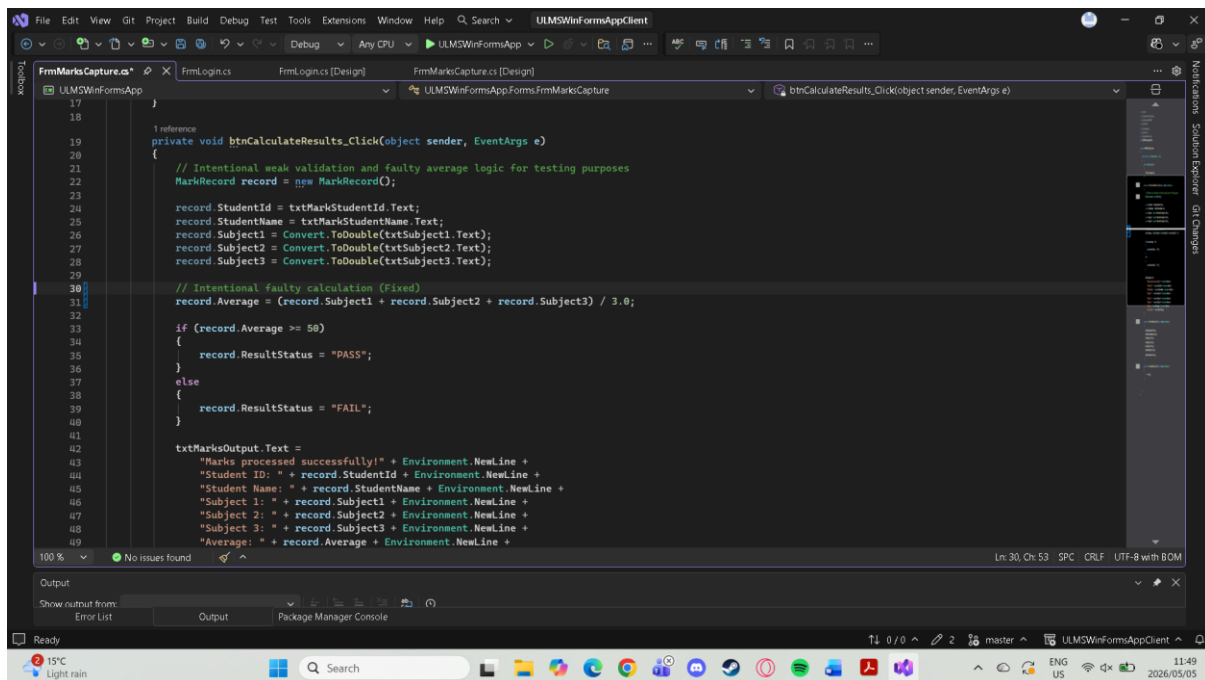


```
30 // Intentional faulty calculation
31 record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3;
```

Issue Description: Due to operator precedence, division (/) happens before addition. This calculates Subject3 / 3 then adds Subject1 + Subject2. For marks 70, 80, 90 → 70 + 80 + (90/3) = 70+80+30 = 180 instead of (70+80+90)/3 = 80.

Why it Occurs: Missing parentheses around the sum.

Fixed Code:



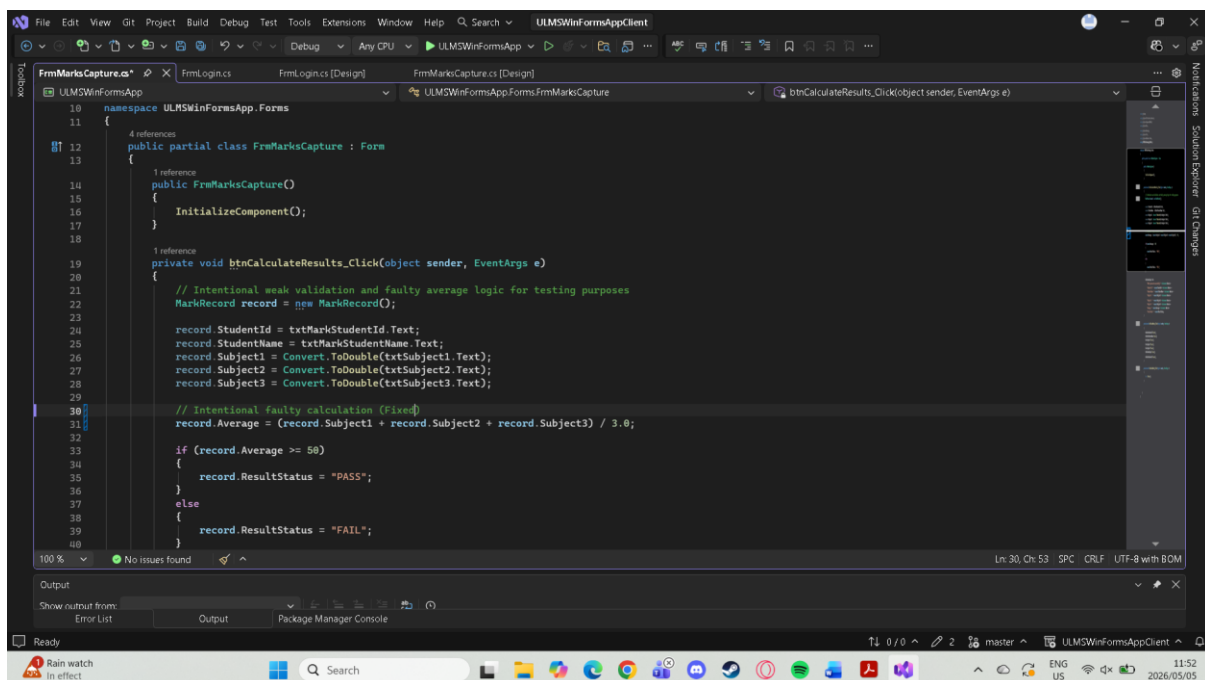
```
17 }
18
19 1 reference
20 private void btnCalculateResults_Click(object sender, EventArgs e)
21 {
22     // Intentional weak validation and faulty average logic for testing purposes
23     MarkRecord record = new MarkRecord();
24
25     record.StudentId = txtMarkStudentId.Text;
26     record.StudentName = txtMarkStudentName.Text;
27     record.Subject1 = Convert.ToDouble(txtSubject1.Text);
28     record.Subject2 = Convert.ToDouble(txtSubject2.Text);
29     record.Subject3 = Convert.ToDouble(txtSubject3.Text);
30
31     // Intentional faulty calculation (Fixed)
32     record.Average = (record.Subject1 + record.Subject2 + record.Subject3) / 3.0;
33
34     if (record.Average >= 50)
35     {
36         record.ResultStatus = "PASS";
37     }
38     else
39     {
40         record.ResultStatus = "FAIL";
41     }
42
43     txtMarksOutput.Text =
44         "Marks processed successfully!" + Environment.NewLine +
45         "Student ID: " + record.StudentId + Environment.NewLine +
46         "Student Name: " + record.StudentName + Environment.NewLine +
47         "Subject 1: " + record.Subject1 + Environment.NewLine +
48         "Subject 2: " + record.Subject2 + Environment.NewLine +
49         "Subject 3: " + record.Subject3 + Environment.NewLine +
50         "Average: " + record.Average + Environment.NewLine +
```

Defect 3

Location: Marks Capture

Issue: No Input Validation, it crashes on non-numeric

Original Faulty Code:

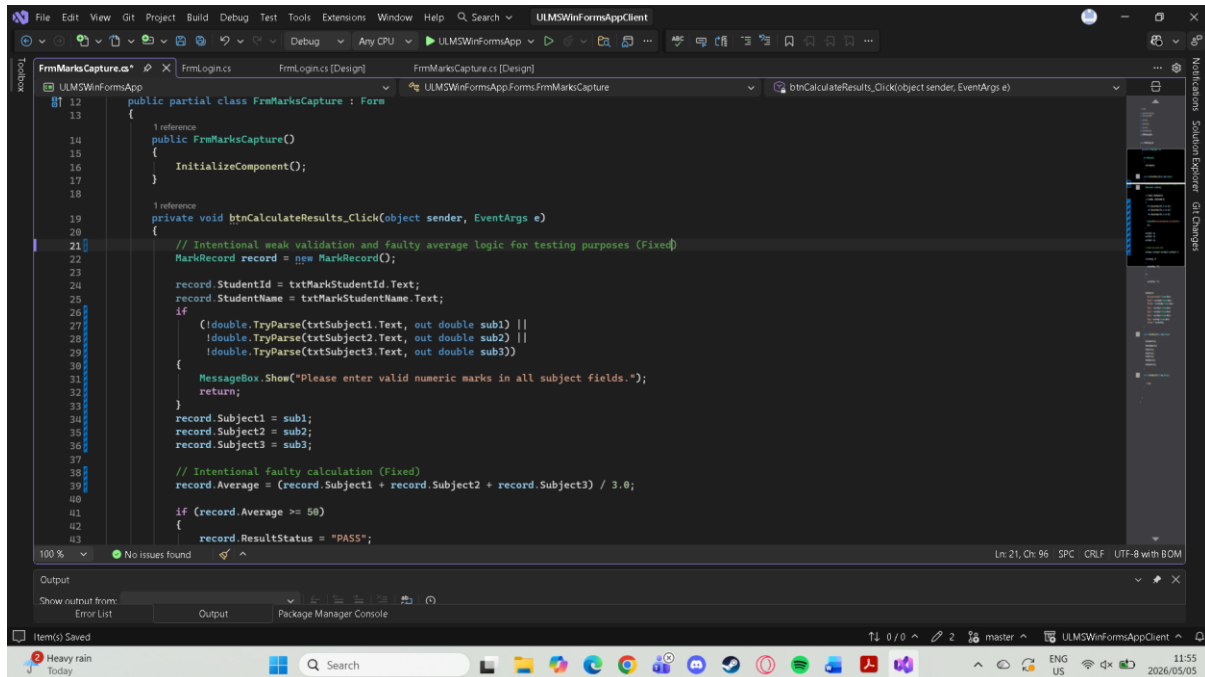


```
10 namespace ULMSWinFormsApp.Forms
11 {
12     4 references
13     public partial class FrmMarksCapture : Form
14     {
15         1 reference
16         public FrmMarksCapture()
17         {
18             InitializeComponent();
19         }
20
21         1 reference
22         private void btnCalculateResults_Click(object sender, EventArgs e)
23         {
24             // Intentional weak validation and faulty average logic for testing purposes
25             MarkRecord record = new MarkRecord();
26
27             record.StudentId = txtMarkStudentId.Text;
28             record.StudentName = txtMarkStudentName.Text;
29             record.Subject1 = Convert.ToDouble(txtSubject1.Text);
30             record.Subject2 = Convert.ToDouble(txtSubject2.Text);
31             record.Subject3 = Convert.ToDouble(txtSubject3.Text);
32
33             // Intentional faulty calculation (Fixed)
34             record.Average = (record.Subject1 + record.Subject2 + record.Subject3) / 3.0;
35
36             if (record.Average >= 50)
37             {
38                 record.ResultStatus = "PASS";
39             }
40             else
41             {
42                 record.ResultStatus = "FAIL";
43             }
44         }
45     }
46 }
```

Issue description: If any textbox is empty or contains non-numeric text, Convert.ToDouble throws FormatException, crashing the application.

Why It Occurs: No TryParse or validation before conversion.

Fixed Code:



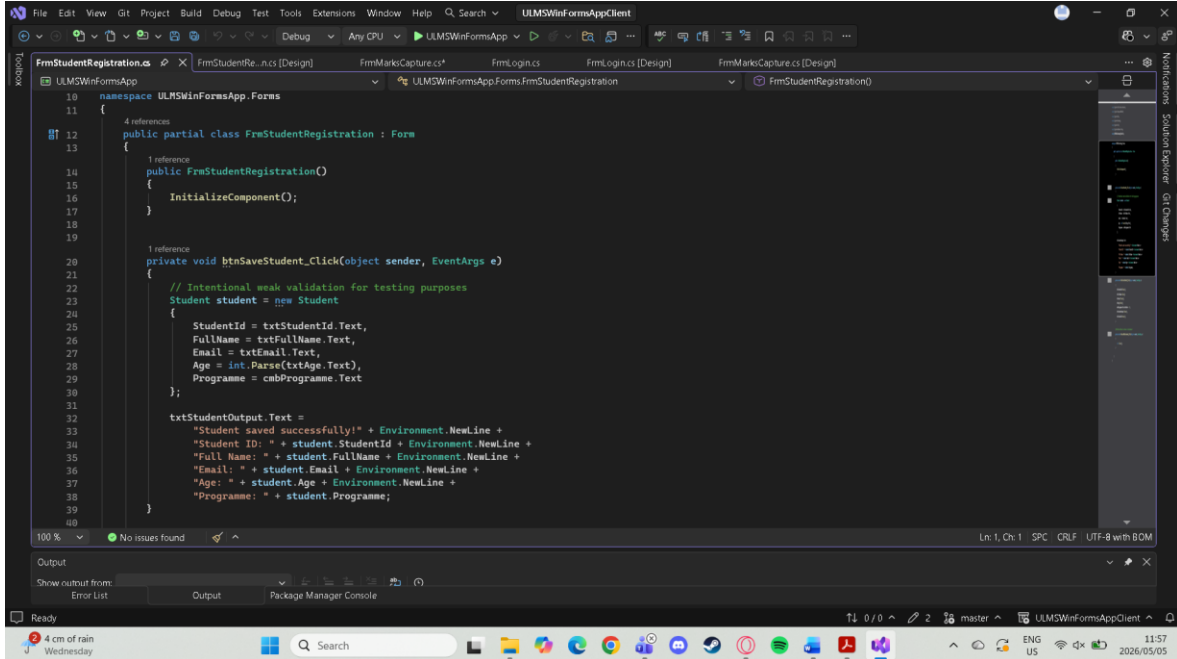
```
12 public partial class FrmMarksCapture : Form
13 {
14     1 reference
15     public FrmMarksCapture()
16     {
17         InitializeComponent();
18     }
19
20     1 reference
21     private void btnCalculateResults_Click(object sender, EventArgs e)
22     {
23         // Intentional weak validation and faulty average logic for testing purposes (Fixed)
24         MarkRecord record = new MarkRecord();
25
26         record.StudentId = txtMarkStudentId.Text;
27         record.StudentName = txtMarkStudentName.Text;
28         if
29         {
30             (double.TryParse(txtSubject1.Text, out double sub1) ||
31             !double.TryParse(txtSubject2.Text, out double sub2) ||
32             !double.TryParse(txtSubject3.Text, out double sub3))
33         {
34             MessageBox.Show("Please enter valid numeric marks in all subject fields.");
35             return;
36         }
37         record.Subject1 = sub1;
38         record.Subject2 = sub2;
39         record.Subject3 = sub3;
40
41         // Intentional faulty calculation (Fixed)
42         record.Average = (record.Subject1 + record.Subject2 + record.Subject3) / 3.0;
43
44         if (record.Average >= 50)
45         {
46             record.ResultStatus = "PASS";
47         }
48     }
49 }
```

Defect 4

Location: Student Registration

Issue: Age Validation Missing

Original Faulty Code:

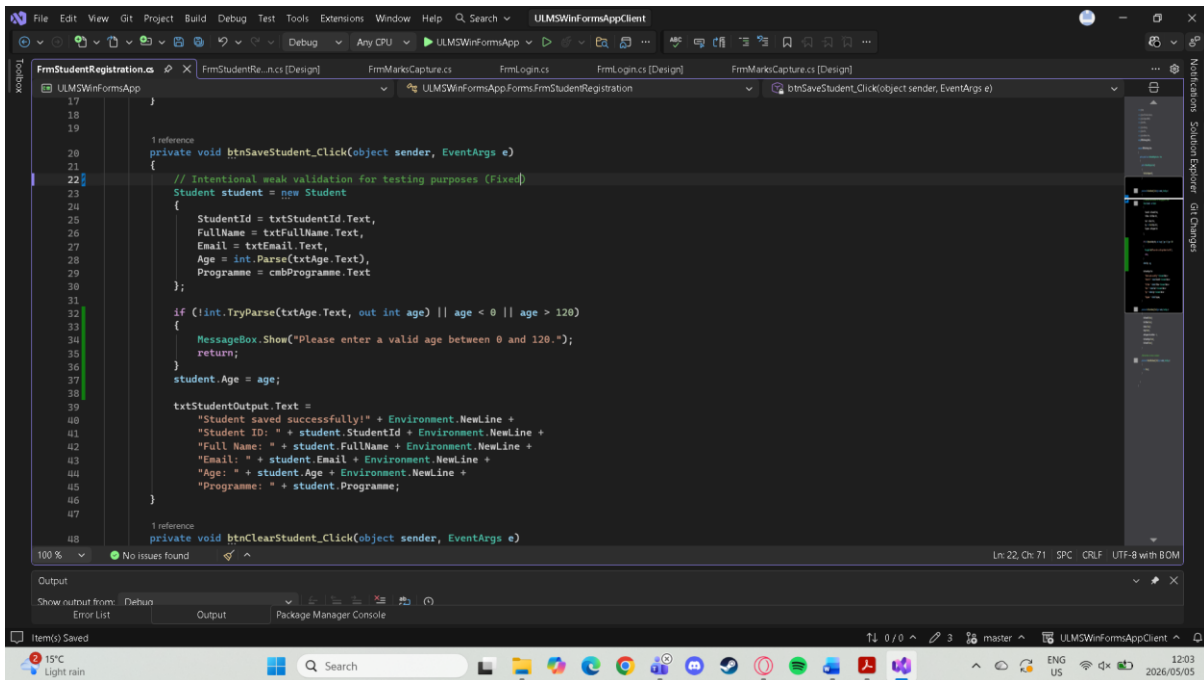


```
10 namespace ULMSWinFormsApp.Forms
11 {
12     public partial class FrmStudentRegistration : Form
13     {
14         public FrmStudentRegistration()
15         {
16             InitializeComponent();
17         }
18
19
20         private void btnSaveStudent_Click(object sender, EventArgs e)
21         {
22             // Intentional weak validation for testing purposes
23             Student student = new Student
24             {
25                 StudentId = txtStudentId.Text,
26                 FullName = txtFullName.Text,
27                 Email = txtEmail.Text,
28                 Age = int.Parse(txtAge.Text),
29                 Programme = cmbProgramme.Text
30             };
31
32             txtStudentOutput.Text =
33                 "Student saved successfully!" + Environment.NewLine +
34                 "Student ID: " + student.StudentId + Environment.NewLine +
35                 "Full Name: " + student.FullName + Environment.NewLine +
36                 "Email: " + student.Email + Environment.NewLine +
37                 "Age: " + student.Age + Environment.NewLine +
38                 "Programme: " + student.Programme;
39         }
40     }
41 }
```

Issue description: No validation – if user enters non-numeric or negative age, int.Parse throws exception. Also, no age range check (e.g., 0-120).

Why It Occurs: Developer assumed numeric input only.

Fixed Code:



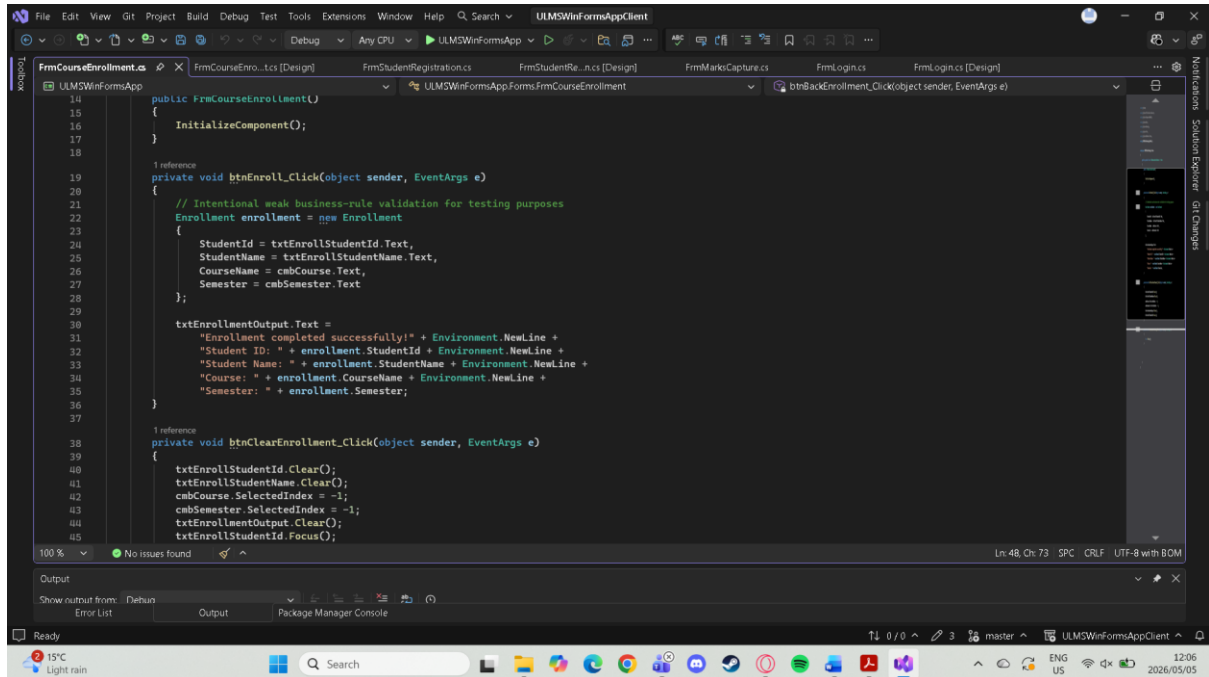
```
17
18
19
20         private void btnSaveStudent_Click(object sender, EventArgs e)
21         {
22             // Intentional weak validation for testing purposes (Fixed)
23             Student student = new Student
24             {
25                 StudentId = txtStudentId.Text,
26                 FullName = txtFullName.Text,
27                 Email = txtEmail.Text,
28                 Age = int.Parse(txtAge.Text),
29                 Programme = cmbProgramme.Text
30             };
31
32             if (!int.TryParse(txtAge.Text, out int age) || age < 0 || age > 120)
33             {
34                 MessageBox.Show("Please enter a valid age between 0 and 120.");
35                 return;
36             }
37             student.Age = age;
38
39             txtStudentOutput.Text =
40                 "Student saved successfully!" + Environment.NewLine +
41                 "Student ID: " + student.StudentId + Environment.NewLine +
42                 "Full Name: " + student.FullName + Environment.NewLine +
43                 "Email: " + student.Email + Environment.NewLine +
44                 "Age: " + student.Age + Environment.NewLine +
45                 "Programme: " + student.Programme;
46         }
47     }
48     private void btnClearStudent_Click(object sender, EventArgs e)
49     {
50     }
```

Defect 5

Location: Course Enrolment

Issue: No Duplicate Prevention

Original Faulty Code:

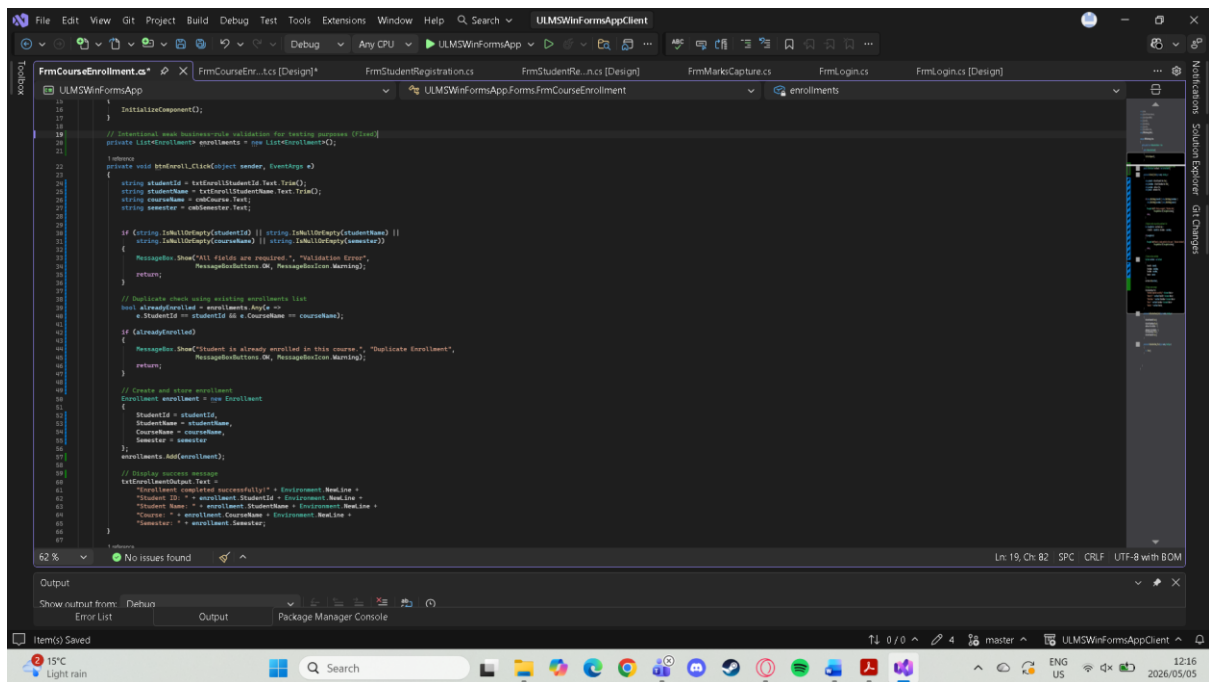


```
14 public FrmCourseEnrollment()
15 {
16     InitializeComponent();
17 }
18
19 1 reference
20 private void btnEnroll_Click(object sender, EventArgs e)
21 {
22     // Intentional weak business-rule validation for testing purposes
23     Enrollment enrollment = new Enrollment
24     {
25         StudentId = txtEnrollStudentId.Text,
26         StudentName = txtEnrollStudentName.Text,
27         CourseName = cmbCourse.Text,
28         Semester = cmbSemester.Text
29     };
30
31     txtEnrollmentOutput.Text =
32         "Enrollment completed successfully!" + Environment.NewLine +
33         "Student ID: " + enrollment.StudentId + Environment.NewLine +
34         "Student Name: " + enrollment.StudentName + Environment.NewLine +
35         "Course: " + enrollment.CourseName + Environment.NewLine +
36         "Semester: " + enrollment.Semester;
37 }
38
39 1 reference
40 private void btnClearEnrollment_Click(object sender, EventArgs e)
41 {
42     txtEnrollStudentId.Clear();
43     txtEnrollStudentName.Clear();
44     cmbCourse.SelectedIndex = -1;
45     cmbSemester.SelectedIndex = -1;
46     txtEnrollmentOutput.Clear();
47     txtEnrollStudentId.Focus();
48 }
```

Issue description: No duplicate check – same student can be enrolled in the same course multiple times. No validation for empty fields.

Why It Occurs: No data structure stores previous enrolments, and no duplicate check is performed before adding a new enrolment.

Fixed Code:



```
16 InitializeComponent();
17 }
18 // Optional: add business rule validation for testing purposes (false)
19 private List<Enrollment> enrollments = new List<Enrollment>();
20
21 //using
22 private void btnEnroll_Click(object sender, EventArgs e)
23 {
24     string studentId = txtEnrollStudentId.Text.Trim();
25     string studentName = txtEnrollStudentName.Text.Trim();
26     string courseName = cmbCourse.Text;
27     string semester = cmbSemester.Text;
28
29     if (string.IsNullOrEmpty(studentId) || string.IsNullOrEmpty(studentName) ||
30         string.IsNullOrEmpty(courseName) || string.IsNullOrEmpty(semester))
31     {
32         MessageBox.Show("All fields are required.", "Validation Error",
33             MessageBoxButtons.OK, MessageBoxIcon.Warning);
34         return;
35     }
36
37     // Check for existing enrollment using existing enrollments list
38     bool alreadyEnrolled = enrollments.Any(e =>
39         e.StudentId == studentId && e.CourseName == courseName);
40
41     if (alreadyEnrolled)
42     {
43         MessageBox.Show("Student is already enrolled in this course.", "Duplicate Enrollment",
44             MessageBoxButtons.OK, MessageBoxIcon.Warning);
45         return;
46     }
47
48     // Create and store enrollment
49     Enrollment enrollment = new Enrollment
50     {
51         StudentId = studentId,
52         StudentName = studentName,
53         CourseName = courseName,
54         Semester = semester
55     };
56     enrollments.Add(enrollment);
57
58     // Display message
59     txtEnrollmentOutput.Text =
60         "Enrollment completed successfully!" + Environment.NewLine +
61         "Student ID: " + enrollment.StudentId + Environment.NewLine +
62         "Student Name: " + enrollment.StudentName + Environment.NewLine +
63         "Course: " + enrollment.CourseName + Environment.NewLine +
64         "Semester: " + enrollment.Semester;
65 }
66
67 }
```


Task 6

Bug ID	Module	Description	Severity	Priority	Status	Resolution
Bug -1	Login	Faulty login logic using instead of && allows unauthorised access	Critical	Highest	Fixed	Changed to && and added empty field validation
Bug -2	Marks Capture	Incorrect average calculation due to missing parentheses (A+B+C/3)	Major	High	Fixed	Changed to (A+B+C)/3.0
Bug -3	Marks Capture	No input validation – non-numeric or empty marks crash application	Critical	High	Fixed	Added double.TryParse validation
Bug -4	Student Registration	Age field – no validation; non-numeric input crashes	Major	High	Fixed	Added int.TryParse and range check
Bug -5	Course Enrolment	No duplicate prevention; multiple same enrollments allowed	Major	Medium	Fixed	Added List<Enrolment> to store enrolments and a duplicate check before adding new enrolment

Task 7

SOFTWARE TESTING REPORT – ULMS

Executive Summary

The ULMS Windows Forms application was tested using the GitHub Repository Source code. 12 test cases were executed; 4 passed, 8 failed due to critical defects including faulty login logic (|| instead of &&), incorrect average calculation, missing input validation (crashes on non-numeric data), and no duplicate enrolment prevention. After debugging, 5 defects were fixed (login logic, average formula, validation for marks and age, duplicate enrolment check). Remaining issues relate to performance (Thread.Sleep) and missing persistence

Testing Scope

- Login authentication
- Student registration
- Course enrolment (UI + in-memory duplicate prevention)
- Marks capture and average calculation

Tools Used

- Visual Studio 2026
- Manual test execution

Defects Identified

Bug ID	Description	Severity	Fixed
Bug-1	Login uses instead of && – unauthorised access	Critical	Yes
Bug-2	Incorrect average formula $A+B+C/3$	Major	Yes
Bug-3	Marks capture – no validation (crash on non-numeric)	Critical	Yes
Bug-4	Student registration – age parsing crash	Major	Yes
Bug-5	Course enrolment – no duplicate prevention	Major	Yes

Corrections Implemented

1. **Login** – changed || to &&, added empty field validation.
2. **Marks average** – corrected formula to $(A+B+C)/3.0$.
3. **Marks input** – added `double.TryParse` to prevent crashes.
4. **Student age** – added `int.TryParse` and range check.
5. **Course enrolment** – added `List<Enrolment>` in-memory store with duplicate check.

Recommendations

- Remove `Thread.Sleep` from report generation.
- Add persistent data storage (database) for production.
- Improve UI feedback with error providers

Deployment Readiness Decision

DEPLOY for TESTING ENVIRONMENT only

All critical and major functional defects have been fixed. The system now handles invalid inputs gracefully and prevents duplicate enrolments. However, lacking persistent storage and containing an artificial performance delay, it is not production-ready.

Signed:

Josh Jansen Van Rensburg